Adaptive Multi-Agent AI Tutoring System for Personalized Programming Education

Project report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Kataru Reddy Eswar AM.SC.U4CSE23126

Bharath Bommineni AM.SC.U4CSE23112

Chimmili Lokesh AM.SC.U4CSE23114

Ramisetti Gowtham Tej AM.SC.U4CSE23144

Under the guidance of

Ms. Anju Sukumar

[Guide designation]



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS

KOLLAM 690525

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "Adaptive Multi-Agent AI Tutoring System for Personalized Programming Education", submitted by Kataru Reddy Eswar (AM.SC.U4CSE23126), Bharath Bommineni (AM.SC.U4CSE23112), Chimmili Lokesh (AM.SC.U4CSE23114) and Ramisetti Gowtham Tej (AM.SC.U4CSE23144), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Computing, Amritapuri, during Semester 7 of the academic year 2026 - 2027.

Ms. Anju Sukumar	[Coordinator name]
Project Guide	Project Coordinator
Signature _____	Signature _____
[Chairperson name]	
Chairperson, CSE	Reviewers
Signature _____	Signature _____

Place: Amritapuri
Date: [DD Month YYYY]

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

DECLARATION

We, Kataru Reddy Eswar (AM.SC.U4CSE23126), Bharath Bommineni (AM.SC.U4CSE23112), Chimmili Lokesh (AM.SC.U4CSE23114) and Ramisetti Gowtham Tej (AM.SC.U4CSE23144), hereby declare that the project entitled "Adaptive Multi-Agent AI Tutoring System for Personalized Programming Education" is a record of our original work carried out under the guidance of Ms. Anju Sukumar, [Guide designation], Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Amritapuri. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, fellowship or similar qualification to any candidate at any university. All sources and contributions of others have been appropriately acknowledged.

Place: Amritapuri
Date: [DD Month YYYY]

Student details	**Signature of the student**
Kataru Reddy Eswar AM.SC.U4CSE23126	_____
Bharath Bommineni AM.SC.U4CSE23112	_____
Chimmili Lokesh AM.SC.U4CSE23114	_____
Ramisetti Gowtham Tej AM.SC.U4CSE23144	_____

Project Guide: Ms. Anju Sukumar

Signature of the Project Guide: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Ms. Anju Sukumar, Department of Computer Science and Engineering, for her guidance throughout this project. Her questions about how a tutor should decide when, and how much, to help a beginner shaped the design of the scaffolding and reflection components, and her review of our progress at each stage kept the work focused on learners rather than on technology alone.

We thank the Project Coordinator and the Chairperson of the Department of Computer Science and Engineering for organising the project reviews and for their feedback, and the faculty members of the department for the courses in programming, databases, software engineering and artificial intelligence on which this work builds.

We are grateful to Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri Campus, for the academic environment and facilities that made this project possible.

We also acknowledge the authors and maintainers of the open-source software used in this project, including Python, FastAPI, LangGraph, PostgreSQL, ChromaDB, React and Docker, and the researchers whose published work is reviewed and cited in this report.

Finally, we thank our families and friends for their encouragement and patience during the course of this work.

Kataru Reddy Eswar
Bharath Bommineni
Chimmili Lokesh
Ramisetti Gowtham Tej

ABSTRACT

Large language model (LLM) assistants can produce a complete answer to a beginner's programming exercise in seconds, but a learner who copies that answer does not build the reasoning the exercise was meant to develop. Most LLM tutors also keep no lasting model of the learner. This project designs and implements CodePath, a web-based tutor for novice Python programmers built on the Cognitive-State Guided Multi-Agent Adaptive Scaffolding Framework (CSMASF). After every run, submission or help request, six cooperating agents run in a cyclic LangGraph workflow. A Diagnostic agent analyses test results and the program's syntax tree. A Cognitive State agent estimates topic mastery with Bayesian Knowledge Tracing, together with confusion, workload and progress. A Similar Learner Retrieval agent finds past situations by cosine similarity in ChromaDB. An Adaptive Scaffolding agent applies the Dynamic Scaffolding Decision Algorithm (DSDA) to decide whether to help, at which of five levels and with which strategy, then generates and validates the hint. A Reflection agent asks about the learner's own code and replies to the answer, and a Learning Planner sequences practice by technique and difficulty. Help never contains a complete solution, does not repeat, and grows only when the learner asks. The system includes 496 verified problems across 17 topics with 15,717 tests. On the final code, 551 backend tests, 59 browser tests and 21 frontend unit tests passed, and an audit that drove nine real scenarios through all agents passed all 37 of its behavioural checks. No study with real learners has been carried out yet, so effects on learning remain to be measured. The project contributes to United Nations Sustainable Development Goal 4, Quality Education, through Target 4.4, by helping novices acquire programming skills relevant to employment while keeping them responsible for their own reasoning.

Keywords: Intelligent tutoring systems; Multi-agent systems; Adaptive scaffolding; Knowledge tracing; Programming education; Large language models

Table of Contents

Acknowledgement i

Abstract ii

List of Figures v

List of Tables vi

1 Introduction 1

1.1 Background and motivation 1

1.2 Objectives 2

1.3 Scope and constraints 4

1.4 Project report organization 4

2 Problem Definition 6

2.1 Problem statement 6

2.2 Existing system and limitations 7

3 Literature Review 9

3.1 Review scope and selection 9

3.2 Descriptive review of literature works 9

3.3 Comparison of reviewed literature 17

4 Requirements 20

4.1 Functional requirements 20

4.2 Non-functional requirements 22

4.3 Hardware requirements 24

4.4 Software and data requirements 24

5 Proposed Methodology **27**

5.1 Design (architecture) or conceptual framework 27

5.2 Proposed approach and procedure 28

6 Results and Analysis **42**

6.1 Evaluation setup 42

6.2 Evaluation criteria 43

6.3 Result 44

6.4 Comparative Analysis 50

6.5 Discussion and Interpretation 54

6.6 Limitations 55

7 Conclusion **56**

7.1 Conclusion 56

7.2 Future work 57

References **58**

A Source Code and Reproducibility **61**

A.1 Setup and execution 61

A.2 Supporting material 62

List of Figures

5.1 System architecture of CodePath 27

5.2 The cyclic learning workflow implemented as a LangGraph graph 29

5.3 Decision flow of the Dynamic Scaffolding Decision Algorithm 35

5.4 Core entities of the data model (simplified; lines show references) 40

6.1 Backend tests per test file (551 in total, all passed) 46

6.2 Agent audit: checks passed per scenario (39 of 39) 46

6.3 First wrong submission: the tutor gives one targeted hint built from the learner's own failing call, values and loop variable, with the concept, a link to the lesson and a "Why this hint?" explanation 47

6.4 Second wrong submission on the same issue: the next hint is offered, not shown, and says once what it will focus on 47

6.5 After three failed attempts: a reflection question about the learner's own loop, and the tutor's reply to the answer "no idea" 48

6.6 After the solve: a question about the technique, and a next step on the same technique without hints, with the route ahead 48

6.7 Home screen after the session: the same next step as the plan, with the route through the course 49

6.8 Teacher dashboard with the clearly labelled synthetic demonstration class; the numbers are generated and are not evidence that the tutor works 49

6.9 Mastery estimates produced by the implemented weighted knowledge-tracing update for three attempt sequences 50

6.10 Curriculum problems per topic and difficulty (496 in total) 51

6.11 Hidden tests by kind of input (14,717 in total) 51

A.1 A topic lesson, opened from the problem's "Read the lesson" link 64

A.2 The practice library, grouping problems by topic in learning order 64

List of Tables

1.1 Objectives, evaluation methods, deliverables and status 3

3.1 Comparison of reviewed literature 17

4.1 Functional requirements 20
4.2 Non-functional requirements 23
4.3 Software used and versions in the tested environment 25

5.1 Isolation applied to each code-execution container 30
5.2 Main diagnostic rules 31
5.3 Evidence weight of each kind of attempt 33
5.4 Help levels and presentation limits 36
5.5 Strategy ladders (level in parentheses) 36

6.1 Evaluation criteria 43
6.2 Results of the automated test suites on the final code 44
6.3 Representative audit checks and recorded evidence 44
6.4 Behaviour before and after the agent improvements 52
6.5 Capabilities of CodePath and closely related systems 53

A.1 Backend test files 63

Chapter 1

Introduction

1.1 Background and motivation

Programming is now a basic skill across engineering and the sciences, yet first programming courses remain difficult for many students. A beginner who is stuck needs help that is timely and matched to what they already understand. Human tutoring is the classic way to provide this, but it does not scale to large classes. Intelligent tutoring systems (ITS) were developed to fill that gap; a widely cited review by VanLehn found that step-based tutoring systems raised learning outcomes by an effect size of 0.76 against no tutoring, close to the 0.79 of human tutoring [16].

Two ideas from learning science explain why the *amount* of help matters as much as its correctness. Wood, Bruner and Ross described *scaffolding*: a tutor controls the parts of a task that are beyond the learner, so that the learner can complete the rest, and withdraws that support as competence grows [14]. Sweller's cognitive load theory shows that problem solving places heavy demands on working memory, so help that is too long or arrives too often can itself get in the way of learning [15]. Research on help seeking in tutoring systems adds a caution: learners do not always use on-demand help well, and teaching them to seek help more deliberately does not automatically improve learning [4].

Large language model (LLM) assistants have changed the situation. They answer almost any programming question instantly, but they give everyone the same kind of help and readily produce complete solutions. In an eleven-week classroom study of the ChatGPT-based tutor PyTutor, the authors found clear benefits for students with weak foundations and also a risk of over-reliance that may hinder independent problem solving [8]. Tools such as CodeHelp and CodeAid add guardrails so that the assistant does not give away solutions [6, 7], but the published descriptions of these tools do not include a lasting model of each learner that decides how much help to give.

This project responds to that gap. It builds **CodePath**, a tutor for novice Python program-

mers based on the **Cognitive-State Guided Multi-Agent Adaptive Scaffolding Framework (CSMASF)** proposed in the initial project submission. Six cooperating agents diagnose each attempt, estimate the learner's state, look at what helped in similar situations, decide how much help fits, prompt reflection and plan the next problem. The guiding principle is that the tutor should behave like a patient teacher sitting beside the student: it gives only the help the student needs at that moment, never the full answer, and leaves the thinking to the student.

Relevant Sustainable Development Goal. The project is aligned with United Nations **SDG 4: Quality Education**, "Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all" [17]. The most relevant target is **Target 4.4**: "By 2030, substantially increase the number of youth and adults who have relevant skills, including technical and vocational skills, for employment, decent jobs and entrepreneurship." The mapping is as follows.

- **Problem:** programming skill is in demand, but beginners who receive complete answers from AI tools may not develop it.
- **Objectives:** the tutor adapts the amount of help to each learner and protects the learner's own reasoning (Section 1.2).
- **Contribution:** a working, open, web-based tutor with a verified 496-problem Python curriculum, which can be offered alongside a course at low cost and without a paid model, because every agent has a built-in deterministic mode.
- **Intended beneficiaries:** novice programmers, including learners without access to a personal tutor, and teachers, who receive a class dashboard showing who may need support.

The contribution to Target 4.4 is an *expected* benefit. The system has been built and verified as software, but its effect on learners' skills has not yet been measured (Chapter 6).

1.2 Objectives

The five objectives come from the initial project submission. Table 1.1 links each objective to how it is evaluated in this report, its deliverable and its status at the time of writing.

Table 1.1: Objectives, evaluation methods, deliverables and status

No.	Objective	Evaluation method	Deliverable	Status
O1	Build an adaptive tutor as several collaborating agents rather than one conversational model.	Pipeline tests confirm that all six agents run in order and receive each other's outputs; per-stage traces are inspected.	Six agents in a LangGraph workflow inside a web platform.	Achieved.
O2	Continuously estimate mastery, confidence, confusion, cognitive load and progress from behaviour.	Unit and pipeline tests; audit checks that unknown values stay unknown and that estimates react to repeated requests and progress.	Cognitive State agent (method `cs-1.2`).	Achieved as behavioural estimates; not validated against an external measure.
O3	Retrieve similar learners with vector cosine similarity over multidimensional profiles.	Tests of filtering, exclusion of approaches already shown, reliability thresholds and absence of identities in the output.	Retrieval agent over ChromaDB and PostgreSQL (vectors `sv-1`).	Achieved technically; real learner outcomes are not yet available.
O4	Design a scoring algorithm that selects the scaffolding level from mastery, confidence, hint history and response time.	Policy tests, including a documented worked example recomputed by the code; audit checks of escalation and timing.	DSDA, policy version `dsda-2.2`.	Achieved; weights are starting values, not tuned on learners.
O5	Test the framework's effectiveness on programming datasets.	Offline verification of every curriculum problem in the sandbox; scenario audit of all agents; planned learner study.	Verified curriculum, audit tool and evaluation plan.	Partly achieved: software behaviour verified; learning effectiveness not yet measured.

Two further objectives follow from the project's principle and are evaluated in the same way: (i) the tutor never reveals a complete solution, reference solutions or hidden tests, and (ii)

help is not repeated and does not increase unless the learner asks for it.

1.3 Scope and constraints

Intended users. Novice and early intermediate learners of Python, typically first-year undergraduates, and the teachers of such courses.

Included functions.

- A browser workspace with an editor, separate Run, Submit and Hint actions, test results, and saved drafts.
- A curriculum of 17 topics with lessons, 74 techniques and 496 problems.
- The six tutoring agents, typed questions about selected lines, reflection questions and replies, and a learning plan with a route ahead.
- Learner progress and profile pages, and a teacher dashboard.
- Adding a question by pasting it or from a link, with suggested tests that are always labelled as suggestions.

Exclusions.

- Languages other than Python are not offered.
- No model is trained; every agent is rule-based, and an LLM is optional and used only for wording.
- No study with real learners has been carried out within this phase.
- The large IBM Project CodeNet archive was not imported; only its importer exists.

Assumptions. Learners use a modern desktop or mobile browser. The server host can run Docker containers for code execution.

Constraints.

- Development and testing used one laptop with 8 GB of memory, so test suites were run one at a time.
- The code-execution sandbox was tested on Docker Desktop for macOS and has not been independently audited.
- The only available model key had no credit, so the LLM path was tested with a scripted stand-in.

1.4 Project report organization

Chapter 2 defines the problem and the limitations of existing systems. Chapter 3 reviews thirteen research works and compares them. Chapter 4 lists the functional and non-functional

requirements and the hardware, software and data used. Chapter 5 presents the architecture and explains each agent, its inputs, outputs and algorithm. Chapter 6 describes the evaluation setup and criteria, reports the results, compares them with existing approaches and states the limitations. Chapter 7 concludes and outlines future work. The appendix gives the steps to set up and reproduce the system.

Chapter 2

Problem Definition

2.1 Problem statement

A beginner working on a programming exercise repeatedly writes code, runs it, sees it fail and tries again. At each of these moments a good tutor decides several things at once: what exactly went wrong in this version of the code, how much the learner already understands, whether to step in at all, and if so with how much help. A tutor that gives too little help leaves the learner stuck and frustrated; a tutor that gives too much, or gives the answer, removes the thinking the exercise was meant to cause.

The problem addressed by this project is to build a tutor that makes these decisions automatically, consistently and transparently for every learner action, while never handing over a complete solution.

Inputs. For each learner action the system receives:

- the action itself: Run (try the code on visible examples), Submit (graded against all tests), Ask for a hint (optionally "just a small nudge" or "I need more help"), or a typed question about selected lines;
- the exact version of the learner's code and the problem it belongs to;
- the stored history of this learner on this problem and topic: earlier attempts and their results, hints already shown and whether they were followed by progress, reflection answers, and time spent.

Expected outputs.

- a diagnosis of the attempt, with the evidence that supports it;
- an updated estimate of the learner's state, keeping uncertainty and "unknown" where evidence is missing;
- a help decision: wait, offer help, or show help, at a stated level and with a stated

strategy and reason;

- when help is shown, a hint that is specific to the learner's own code, different from earlier hints and free of solution code;
- when useful, one reflection question about the learner's own reasoning, and a reply to the answer;
- a recommendation of what to do next, with the route ahead.

Stakeholders.

- **Learners,** who need help that keeps them thinking.
- **Teachers,** who need to know which learners may need a conversation, without reading every submission.
- **The institution,** which needs a tool that is safe to run on its servers and protects learners' data.

Consequences if unsolved. Learners who rely on instant complete answers may pass exercises without being able to solve similar problems alone; the PyTutor study explicitly reported this risk of over-reliance [8]. Teachers in large courses cannot give every learner timely individual help [6]. Running untrusted student code without isolation would expose the server.

2.2 Existing system and limitations

Four kinds of existing solutions are relevant. Their limitations, as reported in the reviewed literature, define the gap this project addresses.

General-purpose LLM assistants. These are conversational question-and-answer tools. They are always available and fluent, but they keep no model of the individual learner, apply the same help to everyone, and readily produce complete code.

LLM programming assistants with guardrails. CodeHelp [6], CodeAid [7] and RAGMan [9] restrict the assistant so that it explains, gives pseudo-code or answers within a course scope instead of writing solutions. They were deployed in real courses with positive student feedback. Their published descriptions, however, do not include a persistent learner model that decides *how much* help each learner should receive, and RAGMan's retrieval draws on course material rather than on what helped similar learners. PyTutor provides a structured ladder of hints but one of its hint types is a coding solution, and its authors observed over-reliance [8].

Classical intelligent tutoring systems. Systems such as the ACT Programming Tutor model the learner explicitly; knowledge tracing estimates the probability that each skill has been learned and drives mastery learning [1]. Conversational tutors such as DeepTutor showed

that tutoring dialogue can be delivered over the web at scale [3]. These systems depend on carefully authored domain models, which are expensive to build for each new subject.

Learner-modelling research. Deep Knowledge Tracing predicts whether a learner will answer the next exercise correctly more accurately than earlier models [2], and newer work simulates learners or verifies tutoring dialogue [11, 12]. These are prediction, simulation or evaluation methods; on their own they do not decide when and how much to help a real learner in a live programming session.

The gap. Among the reviewed works, no single system combines (i) a continuously updated, explainable learner state, (ii) an explicit decision about whether and how much to help that escalates only on request and never reveals a solution, (iii) retrieval of what helped in similar learners' situations, and (iv) reflection and next-step planning that react to the learner's own answers, all for live programming practice. A survey of LLM applications in programming education likewise concludes that interventions work best with scaffolding and human oversight, and that fully automated approaches often miss pedagogical nuance [13]. CSMASF is designed to fill this gap within an open, testable system.

Chapter 3

Literature Review

3.1 Review scope and selection

The review covers research on learner modelling, tutoring and hint provision for programming, LLM-based programming assistants, and multi-agent or conversational tutors. Sources were searched in the ACM Digital Library, the ACL Anthology, arXiv, the AAAI and IJCAI proceedings, and the Springer and Elsevier journal platforms, using combinations of the keywords *intelligent tutoring system*, *knowledge tracing*, *programming education*, *hint generation*, *LLM programming assistant*, *help seeking*, *scaffolding* and *multi-agent tutoring*.

Works were selected if they (i) were primary research articles in peer-reviewed venues or widely used preprints, (ii) addressed learner modelling, help provision, programming practice or LLM-based tutoring, and (iii) informed a specific design decision in CSMASF. The publication period is 1995 to 2025. Thirteen works were reviewed: the nine listed in the initial project submission, and four added to cover the foundations of knowledge tracing [1], help-seeking research [4], guardrails for LLM help [6] and multi-agent LLM tutoring [10].

Bibliographic details were checked against each work's publisher or index record (DOI, ACL Anthology, IJCAI, arXiv, Crossref or OpenAlex). Findings are reported as stated in each work's abstract or published record, and numerical values are given only where they appear there or, for Deep Knowledge Tracing, in its results table. Where a detail is not stated in the material consulted, this is said explicitly rather than assumed.

3.2 Descriptive review of literature works

The works are presented in order of publication year.

3.2.1 Knowledge tracing

Corbett and Anderson (1995) in "Knowledge tracing: Modeling the acquisition of procedural knowledge" [1] investigate how to model a student's changing knowledge state during skill acquisition, in the context of students learning to write short programs with the ACT Programming Tutor. The work proposes *knowledge tracing*: while the student works, the tutor maintains, for every production rule of its cognitive model of programming (the "ideal student model", grounded in ACT-R theory), an estimate of the probability that the student has learned that rule. The model assumes a two-state view of each rule, learned or not learned, with parameters for initial knowledge, the chance of learning at each opportunity, guessing and slipping. The estimates drive mastery learning: the tutor presents an individualised sequence of exercises until each rule is mastered. The authors assess knowledge tracing by using its estimates to predict students' performance on test exercises, and report that the model predicts that performance well. The approach depends on a hand-authored rule model for each domain, which is costly to build. This work informs the present project directly: the Cognitive State agent uses this two-state update, with fixed guess, slip and learning parameters, as its mastery model, because it is interpretable and needs no training data.

3.2.2 Deep knowledge tracing

Piech et al. (2015) in "Deep Knowledge Tracing" [2] investigate knowledge tracing, the task of modelling a student's knowledge as they work through exercises, for large online learning platforms. The work proposes using recurrent neural networks, specifically long short-term memory (LSTM) networks, which learn from response sequences without expert annotation of the skills each exercise requires. The study uses simulated students, Khan Academy mathematics data (47,495 students, 69 exercise tags, about 1.4 million answers) and the ASSISTments benchmark (15,931 students, 124 exercise tags, about 526 thousand answers). The authors evaluate prediction of the next response with the area under the ROC curve (AUC), using five-fold cross-validation on the real datasets. They report an AUC of 0.85 on the Khan data against 0.68 for standard Bayesian Knowledge Tracing (BKT), and 0.86 on ASSISTments against the best previously reported 0.69, a 25% gain; on simulated data the model reached 0.82 against 0.54 for BKT. The main limitation for tutoring is that the model predicts performance but does not itself decide what help to give, and its internal state is less directly interpretable than BKT's parameters. This work informs the project through its baseline: BKT was kept for its interpretability, and a learned model is a future option once real interaction data exists.

3.2.3 DeepTutor

Rus, Niraula and Banjade (2015) in "DeepTutor: An Effective, Online Intelligent Tutoring System That Promotes Deep Learning" [3] investigate how to deliver instruction tailored to each learner through educational technology. The work proposes DeepTutor, a conversational intelligent tutoring system delivered as a web application that learners can use at any time from a browser-connected device. The study is situated in large-scale experiments with high-school students. The authors evaluate the system through these experiments and report them as evidence that conversational intelligent tutoring can work at scale over the web. Detailed metrics are not given in the published abstract. The main limitations relative to the present project are that the system predates current LLMs and is not designed for programming practice with code execution. This work informs the project by showing that dialogue-based tutoring can be delivered at scale through the web, which supports CodePath's browser-based design and its conversational reflection step.

3.2.4 Help seeking with intelligent tutors

Aleven, Roll, McLaren and Koedinger (2016) in "Help Helps, But Only So Much: Research on Help Seeking with Intelligent Tutoring Systems" [4] investigate how learners seek and use on-demand help in intelligent tutoring systems. The work proposes the Help Tutor, an agent that gave students real-time, in-context feedback on their help-seeking behaviour, together with a knowledge-engineered, rule-based executable model of help seeking. The study is situated in classroom use of tutoring systems and is complemented by a theoretical analysis and a review of later empirical work. The authors evaluate the Help Tutor in a classroom study using students' help-seeking behaviour and domain learning outcomes. They report that the feedback made students use on-demand help more deliberately, an effect that persisted after the feedback stopped, but that it did not improve domain-level learning. The authors now place less weight on on-demand, principle-based help during problem solving, while still recommending that tutors include it, and describe helping students become better self-regulated learners as an open challenge. This work informs the present project's decision that unrequested help is offered rather than pushed, that help climbs only when the learner asks, and that the help itself is designed to keep the learner thinking.

3.2.5 Project CodeNet

Puri et al. (2021) in "Project CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks" [5] investigate the lack of large, diverse datasets for training AI models on coding tasks. The work proposes CodeNet, a dataset of over 14 million code samples, about 500 million lines of code, in 55 programming languages. The samples come from submissions to online programming problems and are annotated to support code similarity and classification, translation between languages, and runtime and memory improvement; sample input and output tests are provided for 98.5% of the samples. The authors supply preprocessing tools that turn source code into model inputs and evaluate the dataset by reporting baseline results for code classification and code similarity. The dataset contains submissions and their outcomes but no records of tutoring, hints or learner states, so it does not by itself support learner modelling or tutoring. This work informs the project as a candidate data source for offline evaluation: CodePath includes a CodeNet importer that aggregates problem statistics, tested on a fixture, but the archive itself was not imported in this phase.

3.2.6 CodeHelp

Liffiton, Sheese, Savelka and Denny (2023) in "CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes" [6] investigate how to give students in large computing classes timely help without encouraging over-reliance on LLMs. The work proposes CodeHelp, an LLM-based tool with guardrails that helps students without revealing solutions, with instructor-facing features and a sequence of prompting strategies that keep outputs suitable for students. The study is situated in a first-year computer and data science course with 52 students, whose interactions were collected over 12 weeks. The authors evaluate usage patterns and student perceptions and include the instructor's reflections. They report that students valued the tool's availability and its help in resolving errors, and that the tool was easy for instructors to deploy and complemented rather than replaced their support. Relative to the present project, the published description does not include a per-learner model that adapts the amount of help over time. This work informs CodePath's rule that help never contains a solution, and its validator, which checks every hint before it is shown.

3.2.7 CodeAid

Kazemitabaar et al. (2024) in "CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs" [7] investigate how an LLM assistant can give students helpful, technically correct support without revealing code solutions. The work proposes CodeAid, which answers conceptual questions, produces pseudo-code with explanations and suggests fixes for incorrect student code. The study is a 12-week deployment in a course with 700 students, analysing about 8,000 uses together with weekly surveys and 22 student interviews, followed by interviews with eight educators. From these the authors derive four design considerations: exploit what AI uniquely offers, simplify query formulation while promoting cognitive engagement, avoid direct answers while encouraging motivated learning, and keep the assistant transparent and controllable for students. Relative to the present project, the published description does not include a continuous learner model or retrieval of what helped similar learners. This work informs CodePath's typed-question feature, which answers from what the selected lines actually did, and its emphasis on transparency through a "Why this hint?" explanation.

3.2.8 PyTutor

Yang, Lin, Lin and Ogata (2024) in "Enhancing Python Learning with PyTutor: Efficacy of a ChatGPT-Based Intelligent Tutoring System in Programming Education" [8] investigate whether a ChatGPT-based tutoring system can help beginners learn Python. The work proposes PyTutor, which gives continuous guidance, problem-solving hints and code explanations, with a structured set of hints for each question covering pseudocode, cloze, basic and advanced coding solutions. The study is an eleven-week experiment in which 35 students used PyTutor and 36 did not. The authors compare engagement, completion and success rates in in-class and after-class exercises. They report that PyTutor was especially effective for students with weak foundations, who showed higher engagement, completion and success, and they also identify a risk of over-reliance that may hinder independent problem-solving skills, recommending balanced use. A single LLM generates the help, and the hint set includes coding solutions. This work informs the project in two ways: it supports a graded ladder of help, and its over-reliance finding motivates CodePath's rules that no level contains a solution and that help is offered rather than pushed.

3.2.9 Integrating AI tutors in a programming course

Ma, Krone Martins and Lopes (2024) in "Integrating AI Tutors in a Programming Course" [9] investigate how to provide course-specific AI tutoring for programming homework. The work proposes RAGMan, an LLM-powered system that supports course-specific and homework-specific tutors using retrieval-augmented generation and strict instructions. The study is situated in an introductory programming course with 455 students, where five homework-specific tutors were offered as an optional resource. The authors evaluate usage, answer accuracy within the intended scope and student perceptions. They report that about half of the students used the tutors, mostly for legitimate homework questions; that within scope the tutors answered accurately 98% of the time; that 78% of the students who used them said they helped their learning; and that students valued the non-judgmental environment. The retrieval draws on course material, and the outcomes are largely self-reported. This work informs the project's retrieval design by contrast: CodePath retrieves what helped in similar *learner situations*, not only course content.

3.2.10 Ruffle&Riley

Schmucker, Xia, Azaria and Mitchell (2024) in "Ruffle&Riley: Insights from Designing and Evaluating a Large Language Model-Based Conversational Tutoring System" [10] investigate how LLMs can lower the cost of authoring conversational tutoring systems. The work proposes a system that automatically generates an editable tutoring script from a lesson text and runs a learning-by-teaching format in which two LLM agents play a student and a professor. The study uses biology lessons in two between-subject online studies with 200 participants in total, comparing the system with simpler question-answering chatbots and a reading activity. The authors report that users found the system engaging and helpful and felt they understood the material, and that they spent more time on the activity, but that short-term learning gains did not differ significantly from the reading activity. The domain is not programming. This work informs the project in two ways: it shows that several cooperating agents can structure a tutoring interaction, and it cautions that engagement and perceived helpfulness are not evidence of learning, which is why this report does not claim learning gains without a learner study.

3.2.11 CoderAgent

Zhan et al. (2025) in "CoderAgent: Simulating Student Behavior for Personalized Programming Learning with Large Language Models" [11] investigate how to evaluate personalised programming tutoring when real programming data is scarce and offline evaluation often does not match real learning. The work proposes CoderAgent, an LLM-based agent inspired by the ACT-R cognitive architecture that simulates students' iterative programming processes, modelling both programming knowledge and coding ability, with a Programming Tree of Thought that breaks each step into why, how, where and what. The study uses real-world datasets, including a Python subset of CodeNet and a college-level Java dataset (CSEDM). The authors evaluate the simulation on predicting a student's next modification intention, its location, the next submission and whether it will be accepted, using accuracy, CodeBLEU and AUC. They report that CoderAgent gives interpretable insights into learning trajectories and simulates students accurately. The work simulates learners rather than tutoring them in real time. This work informs the project as a possible way to test tutoring policies offline before a learner study.

3.2.12 TRAVER

Wang, Dai, Zhang, Ma, Li and Chai (2025) in "Training Turn-by-Turn Verifiers for Dialogue Tutoring Agents: The Curious Case of LLMs as Your Coding Tutors" [12] investigate how LLM tutoring agents can proactively guide learners to complete predefined coding tasks. The work proposes Trace-and-Verify (TRAVER), which combines knowledge tracing to estimate the student's knowledge state with turn-by-turn verification to keep the tutor's guidance on track. The authors also introduce DICT, an automatic evaluation protocol based on controlled student simulation and code-generation tests. They report that coding tutoring is challenging and that TRAVER achieves a significantly higher success rate. The evaluation relies on simulated students, and the approach does not include retrieval of similar learners' experience or an explicit decision about how much help to give. This work informs CodePath's combination of a learner-state estimate with a verification step: every hint is checked by a validator before it is shown.

3.2.13 Survey of LLM applications in programming education

Pitts, Hridi and Lekshmi Narayanan (2025) in "A Survey of LLM-Based Applications in Programming Education: Balancing Automation and Human Oversight" [13] investigate how LLMs can scale timely, personalised support for novice programmers when instructor time is limited. The work surveys recent LLM applications in three areas: formative code feedback, assessment and knowledge modelling. The number of reviewed works is not stated in the published abstract. The authors identify recurring design patterns and report that interventions work best when educator expertise complements model output through human-in-the-loop oversight, scaffolding and evaluation, while fully automated approaches often fail to capture the pedagogical nuances of programming education. They call for greater transparency, closer alignment with pedagogy and systems that adapt to varied learning contexts. As a survey, it does not propose or evaluate a system. This work informs CodePath's design choices of keeping the policy, not an LLM, in control of help decisions, explaining every decision, and giving teachers a dashboard.

3.3 Comparison of reviewed literature

Table 3.1 compares the reviewed works. “Not reported” means the detail is not stated in the material consulted; “Not applicable” means it does not apply to that work.

Table 3.1: Comparison of reviewed literature

Work and citation	**Problem or objective**	**Approach and contribution**	**Study context**	**Evaluation criteria**	**Key findings**	**Limitations and gap**
Work 01 [1] Corbett and Anderson, 1995	Model changing knowledge during skill learning	Knowledge tracing: probability each rule is learned, used for mastery learning	ACT Programming Tutor; students writing short programs	Prediction of performance on test exercises	Predicts test performance well	Needs hand-authored rule model per domain
Work 02 [2] Piech et al., 2015	Predict student performance over time	LSTM-based Deep Knowledge Tracing without expert skill labels	Simulated data, Khan Academy, ASSISTments	AUC, 5-fold cross-validation	AUC 0.86 vs 0.69 (ASSISTments); 0.85 vs 0.68 (Khan)	Predicts but does not choose help; less interpretable
Work 03 [3] Rus et al., 2015	Tailor instruction to each learner	DeepTutor, a web-based conversational ITS	Large-scale experiments with high-school students	Experimental evidence at scale	Conversational ITS shown to work at scale over the web	Pre-LLM; not designed for programming

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Work 04 [4] Aleven et al., 2016	Improve learners' use of on-demand help	Help Tutor feedback; rule-based model of help seeking; review	Classroom study with tutoring systems	Help-seeking behaviour; domain learning	More deliberate help use, lasting; no gain in domain learning	Self-regulated help use remains an open challenge
Work 05 [5] Puri et al., 2021	Lack of large AI-for-code datasets	CodeNet: over 14M samples, 55 languages, with tests	Online programming submissions	Baseline classification and similarity results	Test cases for 98.5% of samples	No tutoring or learner-state data
Work 06 [6] Liffiton et al., 2023	Scalable help without over-reliance	CodeHelp: LLM with guardrails, no solutions	First-year course, 52 students, 12 weeks	Usage patterns, perceptions, instructor reflection	Valued availability and error help; easy to deploy	No per-learner adaptive model reported
Work 07 [7] Kazemitabaar et al., 2024	Helpful support without revealing code	CodeAid: conceptual answers, pseudo-code, fix suggestions	12 weeks, 700 students, about 8,000 uses	Usage logs, surveys, 22 student and 8 educator interviews	Four design considerations for AI assistants	No continuous learner model or similar-learner retrieval reported
Work 08 [8] Yang et al., 2024	Help beginners learn Python	PyTutor: ChatGPT ITS with structured hints	11 weeks; 35 users vs 36 non-users	Engagement, completion and success rates	Most effective for weak-foundation students; over-reliance risk	Single LLM; hints include coding solutions

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Work 09 [9] Ma et al., 2024	Course-specific AI tutoring	RAGMan: retrieval-augmented, homework-specific tutors	Intro course, 455 students, 5 tutors	Usage, in-scope accuracy, perceptions	About half used; 98% accurate in scope; 78% said it helped	Retrieves course content, not learner experience; self-report
Work 10 [10] Schmucker et al., 2024	Cheaper authoring of conversational tutors	Ruffle&Riley: scripts from lesson text; two LLM agents	Biology lessons; 2 online studies, 200 participants	Engagement, perceptions, learning gain	Engaging and helpful; no significant learning difference vs reading	Not programming; engagement is not learning
Work 11 [11] Zhan et al., 2025	Evaluate tutoring with scarce data	CoderAgent: ACT-R-inspired LLM simulation of programming	CodeNet Python subset; CSEDM Java	Accuracy, CodeBLEU, AUC on next-step tasks	Interpretable, accurate simulation	Simulation, not live tutoring
Work 12 [12] Wang et al., 2025	Proactive guidance in coding tutoring	TRAVER: knowledge tracing plus turn-by-turn verification; DICT protocol	Simulated students; code-generation tests	Task success rate	Significantly higher success rate	Simulated evaluation; no help-amount decision or retrieval
Work 13 [13] Pitts et al., 2025	Scale personalised novice support	Survey of feedback, assessment and knowledge modelling	Recent LLM programming-education studies	Not applicable (survey)	Human-in-the-loop and scaffolding work best	Not reported (count of works); proposes no system

Chapter 4

Requirements

4.1 Functional requirements

The actors are the **learner**, the **teacher** and the **system** (the tutoring agents). Table 4.1 lists the functional requirements. The acceptance criterion of each is checked by automated tests named in the table or listed in Appendix A; `test_*.py` files are backend tests and "browser" refers to the Playwright end-to-end suite.

Table 4.1: Functional requirements

ID	Requirement	Actor	Input and expected behaviour or output	Acceptance criterion
FR1	Accounts and onboarding	Learner, teacher	Register, sign in, or start a labelled demo session; a short onboarding records experience and goal.	Browser: real accounts work; demo access is labelled; signed-out visitors are sent to sign in.
FR2	Curriculum and lessons	Learner	Browse 17 topics in learning order; each topic has a lesson per technique with runnable examples.	Browser: the path shows position and leaves every topic open; a topic teaches first, then offers problems by technique.
FR3	Code workspace	Learner	Editor with Python highlighting, find and replace, font size and saved drafts.	Browser: drafts survive a refresh; the editor behaves like a code editor.

ID	Requirement	Actor	Input and expected behaviour or output	Acceptance criterion
FR4	Run	Learner	Run code on visible examples or own test cases; not graded.	`test_hint_request_is_not_a_graded_attempt`; browser: Run, Submit and Hint are separate.
FR5	Submit and results	Learner	Grade against public and hidden tests in the sandbox; results name the code version they belong to.	`test_older_results_never_overwrite_newer_state`; duplicate clicks create one attempt.
FR6	Diagnose each attempt	System	Output category, issue label, evidence, likely cause and comparison with the previous version.	`test_diagnosis.py` (69 tests); audit checks on syntax, runtime and logic issues.
FR7	Estimate learner state	System	Update mastery (with range) and confusion, workload and progress, or report "unknown".	`test_unknown_confidence_is_left_out_not_assumed`; `test_cognitive_state_updates_from_repeated_requests`.
FR8	Retrieve similar situations	System	Suggest a level and strategy from similar past situations, or report no reliable match; no identities or code.	`test_retrieval_output_exposes_no_identities_or_code`; `test_retrieval_does_not_suggest_what_this_learner_already_saw`.
FR9	Decide help (DSDA)	System	Decide wait, offer or show; choose level and strategy; never more than one level up; unrequested help after a hint is only offered.	`test_policy.py` (30 tests), including `test_help_never_climbs_the_ladder_without_a_request`.
FR10	Generate and validate hints	System	Write a hint from the learner's own evidence; reject repeats, vague advice, invented lines and solution code; retry or fall back.	`test_repeated_requests_never_show_the_same_hint`; `test_never_returns_the_solution_through_the_ladder`.

ID	Requirement	Actor	Input and expected behaviour or output	Acceptance criterion
FR11	Questions about code	Learner	Ask a typed question or select lines; the answer uses what the lines really did on a visible example.	Browser: asking about selected lines still runs the agents and answers.
FR12	Reflection	System, learner	Ask one question about the learner's own code when useful; reply to the answer without grading.	`test_reflection_and_planner.py` (27 tests); browser: a reflection answer gets a reply.
FR13	Learning plan	System	Recommend the next problem with a reason and show the route ahead; the home screen agrees with the plan.	`test_planner_keeps_unresolved_task_active`; `test_home_agrees_with_the_plan_after_a_first_try_solve`.
FR14	Progress and profile	Learner	Show solved problems, activity, per-topic state with evidence, and improvement measures.	Browser: the profile shows what was done and each topic's learning state.
FR15	Teacher dashboard	Teacher	Show class measures and learners who may need support; open a student's profile.	Browser: a student's numbers match the teacher's; teacher demo labels its data as synthetic.
FR16	Add a question	Learner	Paste a question or give a link; review parsed fields; suggested tests are labelled, never presented as official.	`test_link_import.py` (90 tests); browser Add Question flows.
FR17	Preferences	Learner	Theme, learning mode, automatic hints and editor settings are saved per account.	Browser: theme persists; font size persists across sessions.

4.2 Non-functional requirements

Table 4.2 gives the quality requirements with measurable criteria. Values marked "configured" are limits set in the code, not measured performance.

Table 4.2: Non-functional requirements

ID	**Quality**	**Requirement and measurable criterion**	**Verification**
NFR1	Security of execution	Student code is untrusted. Each job runs in a fresh container with no network, a read-only root filesystem, all capabilities dropped, an unprivileged user, 256 MB memory, one CPU and at most 64 processes (configured).	Execution-service suite (39 tests, including isolation attacks).
NFR2	Solution protection	No response contains a reference solution, hidden test values or hidden-test output; no hint level contains a complete solution.	`test_hidden_tests_never_reach_the_client`; `test_reference_solution_never_reaches_the_client`; validator tests.
NFR3	Privacy	Retrieval returns only aggregates; learners are stored only as a hash in pooled cases; pooling requires consent.	`test_retrieval_output_exposes_no_identities_or_code`; `test_profile_shows_similar_learners_only_as_counts`.
NFR4	Reliability	A failing agent falls back to a safe default; an execution-service failure is never counted as a learner mistake; a repeated request with the same idempotency key creates one record.	`test_service_failure_is_not_treated_as_learner_mistake`; `test_model_failure_uses_labelled_fallback`; idempotency tests.
NFR5	Explainability	Every decision carries reason codes and a learner-facing reason; every stage writes a trace row.	Audit reads each agent's output from its trace.
NFR6	Robustness to manipulation	Instructions hidden in student code cannot change the policy.	`test_prompt_injection_in_code_cannot_change_the_policy`.
NFR7	Accessibility	No violations of automated WCAG 2.0 and 2.1 level A and AA rules on the main screens, in both themes; no sideways scrolling at phone width.	Browser accessibility suite (axe-core) and phone-layout test.

ID	Quality	Requirement and measurable criterion	Verification
NFR8	Fair use	Per-user limits per minute (configured): 30 runs, 15 submissions, 15 hints, 12 sign-in attempts, 20 imports. A saved draft may hold at most 20,000 characters, and code sent to run, submit or ask is limited to 100,000 characters by request validation.	`test_rate_limit` (with the run limit set to two per minute, the third run is refused with status 429); a longer draft is refused with status 413.
NFR9	Responsiveness	Per-test time limit 2 s by default, total job limit at most 25 s, at most 3 concurrent jobs with a queue of 12 (configured).	Execution-service tests; timeout scenario in the audit.
NFR10	Maintainability	All policy weights, thresholds and ladders live in one versioned file (`policy.yaml`); each hint records the policy version.	Policy tests; recorded `policy_version`.

4.3 Hardware requirements

No special hardware is required: no model is trained and an optional LLM is reached over the network.

- **Development and test machine used:** Apple MacBook Air with an Apple M2 processor and 8 GB of memory, running macOS 27.2 with Docker Desktop 29.8.
- **Server (deployment):** a 64-bit machine able to run Docker containers. Because each code job starts a container with a 256 MB memory limit and up to three jobs run at once, memory for these containers comes on top of the database, API and vector store. A GPU is not required.
- **Learner device:** any device with a current web browser; the interface adapts to phone, tablet and desktop widths.

4.4 Software and data requirements

Software. Table 4.3 lists the main software with the versions installed in the tested environment.

Table 4.3: Software used and versions in the tested environment

Component	**Version**	**Role**
Python	3.12.13	Backend and execution service; the sandbox image uses Python 3.12 slim.
FastAPI, Uvicorn	0.162.2, 0.54.0	HTTP API [21].
Pydantic	2.14.3	Typed contracts between agents.
LangGraph, checkpoint-postgres	1.2.20, 3.2.0	Agent graph, pause and resume [19].
SQLAlchemy, Alembic, psycopg	2.1.8, 1.20.1, 3.3.8	Database access and five migrations.
PostgreSQL	16.15	Main database and graph checkpoints.
ChromaDB	1.5.9	Vector store with cosine HNSW index [20, 18].
httpx	0.28.1	Calls to the execution service and optional LLM providers.
Docker	29.8.0	Isolated execution of student code.
Node.js	26.4.0	Frontend build and browser tests.
React, React Router	18.3.1, 6.30.1	Web interface.
TypeScript, Vite	5.8.3, 6.3.5	Typed frontend code and build tool.
CodeMirror	6 (view 6.36.8)	Code editor with Python support.
TanStack Query, Radix UI	5.80.7, 1.x and 2.x	Server state and accessible interface components.
pytest, Vitest, Playwright	9.2.0, 3.2.4, 1.53.1	Backend, unit and browser tests [22].
axe-core for Playwright	4.10.2	Automated accessibility checks [23].

Data. The system uses the following data.

- **Curriculum (authored for the project):** 17 topics, 74 techniques and 496 Python problems (237 easy, 218 medium, 41 hard). Each problem has a statement, starter code, a reference solution kept on the server, and tests.
- **Tests:** 15,717 in all, of which 1,000 are visible examples and 14,717 are hidden. 2,665 were written with the curriculum and 13,052 were generated from each problem's examples or a problem-specific input generator. Generated tests take their expected answers from the reference solution, and every test, written or generated, is checked against it; 487 problems have 30 or more tests (median 32). Each reference solution passes all of its tests, and each starter fails, in the real sandbox.

- **Similar-situation records:** pooled, anonymous outcomes of real hints, stored only for consenting real accounts, plus synthetic demonstration records that are always labelled and can be switched off.
- **IBM Project CodeNet [5]:** an importer that aggregates problem statistics was built and tested on a small fixture. The archive was not imported in this phase.
- **Imported questions:** questions added from links are read only where the site's terms and `robots.txt` permit; for sites that forbid automated access, the link supplies attribution and the learner pastes the statement.

Chapter 5

Proposed Methodology

5.1 Design (architecture) or conceptual framework

CodePath is a three-tier web application with a separate, isolated service for running student code. Figure 5.1 shows its components and how they communicate.

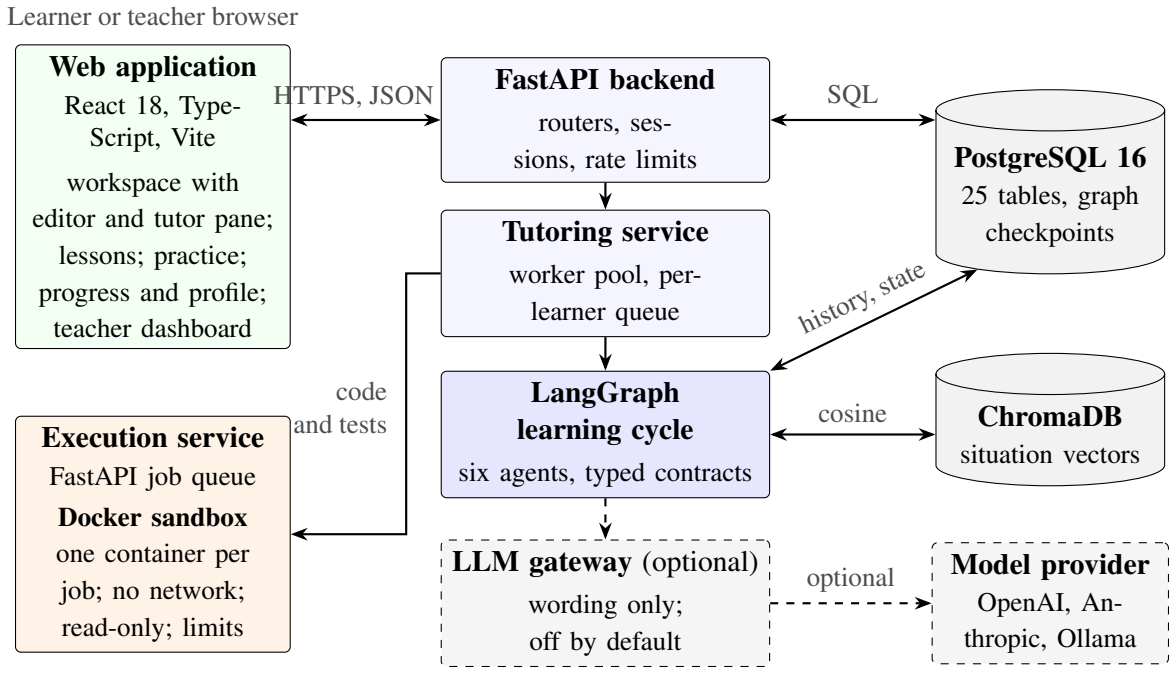


Figure 5.1: System architecture of CodePath

Web application. A single-page React application provides the learner workspace (editor, test cases, results and the tutor pane), the learning path with lessons, the practice library, search, progress and profile pages, settings and the teacher dashboard. It talks to the backend only through a typed JSON API, whose TypeScript types are generated from the backend's OpenAPI schema.

Backend. FastAPI routers handle authentication (Argon2-hashed passwords and server-side

sessions), the workspace (drafts, runs, submissions, hints, questions and reflections), content, imports, learner data and teacher views. The tutoring service queues each learner's actions so that they are processed in order, attaches an idempotency key so that a double click creates one record, and starts the agent pipeline in a worker thread. The browser polls the run and shows the real status of each stage.

Agent pipeline. The six agents run as nodes of a LangGraph graph (Section 5.2.1). Each agent has a typed input and output contract (Pydantic models) and reads the learner's stored history from the database.

Data stores. PostgreSQL holds 25 application tables (Section 5.2.9) and the LangGraph checkpoints, which let a paused learning cycle resume. ChromaDB stores 14-dimensional situation vectors for similar-learner retrieval and searches them with cosine similarity over an HNSW index [20, 18]; the outcomes behind each vector are stored in PostgreSQL.

Execution service. Student code is never run inside the API process. A separate service receives the code and tests over HTTP, starts a fresh, locked-down Docker container per job, runs the tests through a small trusted harness and returns structured results.

Optional LLM gateway. A provider-neutral adapter can call OpenAI, Anthropic or a local Ollama model to reword hints. It is off by default (`LLM_PROVIDER=none`); every caller has a deterministic built-in alternative, so tutoring never depends on a model.

5.2 Proposed approach and procedure

5.2.1 The learning cycle

Every learner action (Run, Submit, Ask for a hint, or a typed question) is processed by the same graph, shown in Figure 5.2. The graph state (`TutorState`) carries only the current cycle; anything earlier is read from the database by a history loader, so the agents remember earlier hints, attempts and answers across requests.

The graph has four conditional edges, taken from the routing functions in `graph.py`:

- after `evaluate`, if the code could not be checked (service failure or cancellation), every agent is recorded as skipped and the cycle goes straight to `finalize`, so a service problem is never treated as a learner mistake;
- after the Cognitive State agent, retrieval is skipped when the problem is solved, because no help needs to be chosen;
- after DSDA, hint generation runs only when the decision is to show help; an offer generates nothing, and accepting it starts a new cycle;
- after validation, a rejected draft returns to generation at the same level and strategy, at most three times in total.

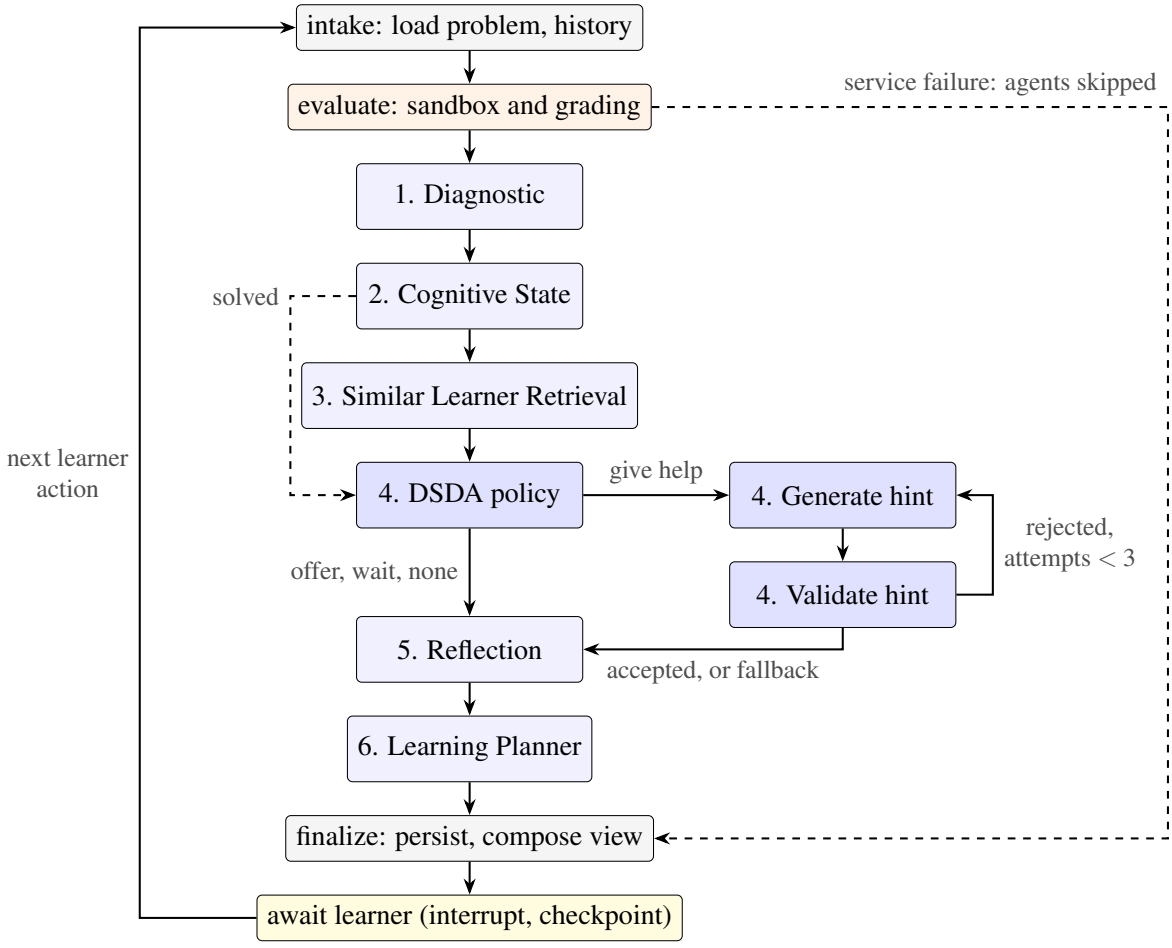


Figure 5.2: The cyclic learning workflow implemented as a LangGraph graph

After `finalize`, the `await_learner` node calls LangGraph's `interrupt`. The state is saved by the PostgreSQL checkpointer under a thread for the learner and problem, and nothing runs until the learner acts again. No loop can generate more help on its own.

Each node runs inside a guard that records an `agent_steps` row (status, input and output summaries, retries, duration) and, if the node fails, substitutes a safe fallback output so the cycle completes.

5.2.2 Code execution and grading

Table 5.1 lists the isolation applied to every job. Inside the container, the harness also sets operating-system resource limits (CPU time, address space, file size and open files) and records a trace of selected lines when a learner asks about them.

Table 5.1: Isolation applied to each code-execution container

Setting	**Effect**
`-network none`	No network access.
`-read-only`, `-tmpfs /tmp` (32 MB, `noexec`)	Read-only root filesystem; a small temporary folder from which programs cannot be executed.
`-memory 256m`, `-cpus 1`, `-pids-limit 64`	Bounded memory, CPU and number of processes.
`-cap-drop ALL`, `no-new-privileges`, `-user 65534`	No Linux capabilities, no privilege escalation, unprivileged user.
`-ipc private`, `-log-driver none`, core dumps off	No shared memory with the host, no logs kept, no core files.
Image `python:3.12-slim` without `pip`	Only the standard library and the trusted harness are available.

Grading compares each returned value with the expected value after converting both to a canonical form. Integers must match exactly; floating-point values are equal within a relative and absolute tolerance of 10^{-9}; Booleans equal only Booleans; lists are compared element by element and dictionaries by keys and values; for tasks where order does not matter both lists are sorted first; printed output is compared after normalising line endings and trailing spaces. Each test receives a category (passed, wrong answer, runtime error, time limit and so on), and the overall verdict is *accepted* if all tests pass, *syntax error* if the code cannot be parsed, *partial* if some tests pass, and otherwise the most serious failure. Inputs, expected values and printed output of hidden tests never leave the server.

5.2.3 Agent 1: Diagnostic

Purpose. Explain what went wrong in this exact version of the code, with evidence, keeping observed facts separate from inferred misconceptions.

Inputs. Verdict and per-test results, the code, the problem (function name, concepts), recent issue labels, the previous attempt and the number of hint requests on the same code.

Outputs. A category, a stable issue label such as `logic:off_by_one`, facts (each tagged with its source: runner, tests or code analysis), inferences with a confidence, affected lines, a confidence value in $[0, 1]$, the repeat count, the failing visible example (input, value returned, value expected), facts about the main loop (its line, source and variable, and the variable it builds up), and a comparison with the previous version.

Method. The agent is rule-based and deterministic. It uses Python's `ast` module to analyse the code's structure without running it, regular expressions over Python's error messages, and `difflib` to find changed lines. Table 5.2 summarises the main rules.

Table 5.2: Main diagnostic rules

Verdict	**Evidence used**	**Issue label**
Syntax error	Python's message, e.g. "expected ':'", "was never closed", indentation errors, unterminated strings, "maybe you meant '==' "; line from Python	`syntax:missing_colon`, `syntax:unbalanced_bracket`, `syntax:indentation`, ...
Time limit	A `while` loop whose condition variables are never updated in its body; a recursive function with no base case	`timeout:loop_no_progress`, `timeout:recursion_no_base`, `timeout:general`
Runtime error	Exception type and message matched against a table of causes; for `NameError` and `UnboundLocalError`, where the name is defined relative to where it is used	`runtime:TypeError:mix_text_number`, `runtime:UnboundLocalError:used_before_assignment`, ...
All answers `None`	Empty body; `print` instead of `return`; no `return`	`logic:not_implemented`, `logic:print_instead_of_return`, `logic:missing_return`
Wrong answer	A `return` as a direct child of a loop body	`logic:early_return_in_loop`

Verdict	Evidence used	Issue label
Wrong answer	Every numeric answer differs by exactly 1, or by exactly one of the input's end values, plus a `range` loop	`logic:off_by_one`
Wrong answer	Same items in a different order; only spacing or capitalisation differs; wrong type	`logic:wrong_order`, `logic:text_format`, `logic:wrong_return_type`

For example, for the code in Figure 6.3, which loops over `range(1, n)`, the call `sum_to_n(3)` returns 3 instead of 6. The difference, 3, equals the input $n = 3$, an end value of the range, so the agent reports `logic:off_by_one` on line 3. The comparison with the previous version records whether the code changed, which lines changed, whether the issue is the same, and whether the new version gets further, using the order syntax error $<$ crash $<$ wrong answer $<$ partial $<$ accepted and the change in tests passed.

5.2.4 Agent 2: Cognitive State

Purpose. Estimate topic mastery, confidence, confusion, cognitive load and progress, each with its evidence, and leave a value unknown when there is not enough evidence.

Inputs. The topic's previous mastery and evidence weight, this result (category, pass ratio, graded or not, same code as before), the problem's difficulty and origin, earlier attempts on the problem, hints used and their highest level, hints not followed by progress, active time, the learner's own ratings, the latest reflection answer and requests on unchanged code.

Mastery: weighted Bayesian Knowledge Tracing. The agent uses the two-state knowledge tracing model of Corbett and Anderson [1]. With prior mastery p, guess probability $g = 0.20$, slip probability $s = 0.10$ and learning probability $t = 0.12$ (fixed in `policy.yaml`), the posterior after an observation is

$$p_{|\text{obs}} = \begin{cases} \dfrac{p(1-s)}{p(1-s)+(1-p)\,g} & \text{if solved,} \\ \dfrac{p\,s}{p\,s+(1-p)(1-g)} & \text{otherwise,} \end{cases} \qquad p_{\text{learn}} = p_{|\text{obs}} + (1-p_{|\text{obs}})\,t. \tag{5.1}$$

Unlike standard BKT, the update is weighted by how much the attempt counts as evidence:

$$p_{\text{new}} = p + w\,(p_{\text{learn}} - p), \tag{5.2}$$

where the weight w is given in Table 5.3. Runs, hint requests and resubmitting unchanged code have $w = 0$ and leave mastery unchanged. The starting value is 0.20. Starting from 0.20, one failed attempt ($w = 0.8$) gives 0.157 and one solve without help ($w = 1$) gives

0.586.

Table 5.3: Evidence weight of each kind of attempt

Attempt	**Weight** w
Solved without hints	1.0
Solved after hints, by highest hint level L0 to L4	1.0, 0.8, 0.6, 0.35, 0.2
Logic failure (wrong answer, partial, runtime error, missing function)	0.8
Time, memory or output limit	0.6
Syntax error	0.15
Imported (learner-added) problem	weight above $\times 0.5$
Run, hint request, or the same code resubmitted	0

The total evidence E is the sum of the weights so far. Mastery is shown with a range $p \pm 0.35/\sqrt{1+E}$, clipped to $[0,1]$, so the range narrows as evidence accumulates, and with a band: "not enough evidence" while $E < 1.5$, then starting ($p < 0.40$), developing (< 0.65), proficient (< 0.85) or strong.

Confusion is estimated from counted signals: the same issue in two or more attempts in a row; a hint not followed by progress; the kind of error changing across three attempts; the learner saying they feel stuck; an unsure reflection answer; two or more requests on unchanged code; a hint marked unhelpful. No history and no signals give *unknown*, because one failure is not evidence of confusion; one signal gives moderate; two or more give high.

Cognitive load counts: active time above twice the usual for the difficulty (easy 4, medium 10, hard 20 minutes); three or more hints on the problem; three attempts in a row passing no tests; the learner saying the task feels like a lot. A single long pause is treated as at most moderate, because it may be a break.

Progress is solved, improving (the code gets further, or more tests pass than the best so far), regressing, stalled (the same issue repeating), steady or unknown. **Confidence** comes only from the learner's own rating and is never inferred.

5.2.5 Agent 3: Similar Learner Retrieval

Purpose. Find past situations similar to the current one and report which kind of help was followed by progress, or report that there is no reliable match.

Situation vector. Eight state values are mapped from $[0,1]$ to $[-1,1]$ by $c(x) = 2(x-0.5)$ and six category slots are added, giving the 14-dimensional vector

$$\mathbf{v} = \big(c(m), c(\kappa), c(f), c(\ell), c(r), c(d), c(\min(a,6)/6), c(\min(h,4)/4), \mathbf{e}\big), \tag{5.3}$$

where m is mastery, κ confidence, f confusion, ℓ load, r progress, d difficulty, a attempts,

h hints and **e** has 0.8 for the current error category (syntax, runtime, wrong answer, partial, timeout, resource) and -0.1 for the others. Centring matters because cosine similarity ignores vector length: without it, a weak and a strong learner would point in the same direction. Similarity is

$$\mathrm{sim}(\mathbf{u},\mathbf{v}) = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}. \tag{5.4}$$

Procedure. ChromaDB returns up to 150 nearest stored situations in the same category group. The agent then reads the corresponding outcome records from PostgreSQL and keeps only cases that belong to other learners (owners are compared by a SHA-256 hash), have similarity of at least 0.72, have a usable outcome, differ in difficulty by at most 0.35, and share the issue or the topic. At least five such candidates are required. Measured ("live") cases are preferred when at least five exist; otherwise labelled synthetic cases may be used. Candidates are grouped by (level, strategy), leaving out strategies this learner has already seen on the issue, and the group with the best cautious success rate is chosen:

$$\hat{p} = \frac{k+1}{n+2}, \qquad \mathrm{LB} = \hat{p} - \sqrt{\frac{\hat{p}(1-\hat{p})}{n+2}}, \tag{5.5}$$

where k of the group's n cases were followed by progress or a solve. This prevents one success out of one from outranking eight out of ten. The chosen group must contain at least three cases. The output gives the suggested level and strategy, the smoothed effectiveness $\hat{p}$, the support n, the provenance (live, synthetic or mixed) and the mean similarity, and never any identity or code.

DSDA uses a suggestion only when its support is at least 3 and its effectiveness at least 0.55, and it may move the level by at most one. Similarity is not treated as proof that a kind of help will work.

5.2.6 Agent 4: Adaptive Scaffolding (DSDA)

The Dynamic Scaffolding Decision Algorithm separates three steps: a *policy* decides whether and how much to help, *generation* writes the words, and *validation* checks them. An LLM, when configured, may only write the words; it cannot change the level, the strategy, the length limits, the solution constraint or the decision to wait. Figure 5.3 summarises the flow.

Support score. Let K be the set of inputs whose value is known. The score is a weighted mean over K, so an unknown value never counts as a number:

$$S = \mathrm{clip}_{[0,1]}\Big(\sum_{k\in K}\frac{w_k}{\sum_{j\in K} w_j}\,x_k \;-\; 0.15\cdot[\text{improving}]\Big). \tag{5.6}$$

The inputs $x_k \in [0,1]$ and weights w_k are: low mastery $1-m$ (0.5 with no evidence), weight

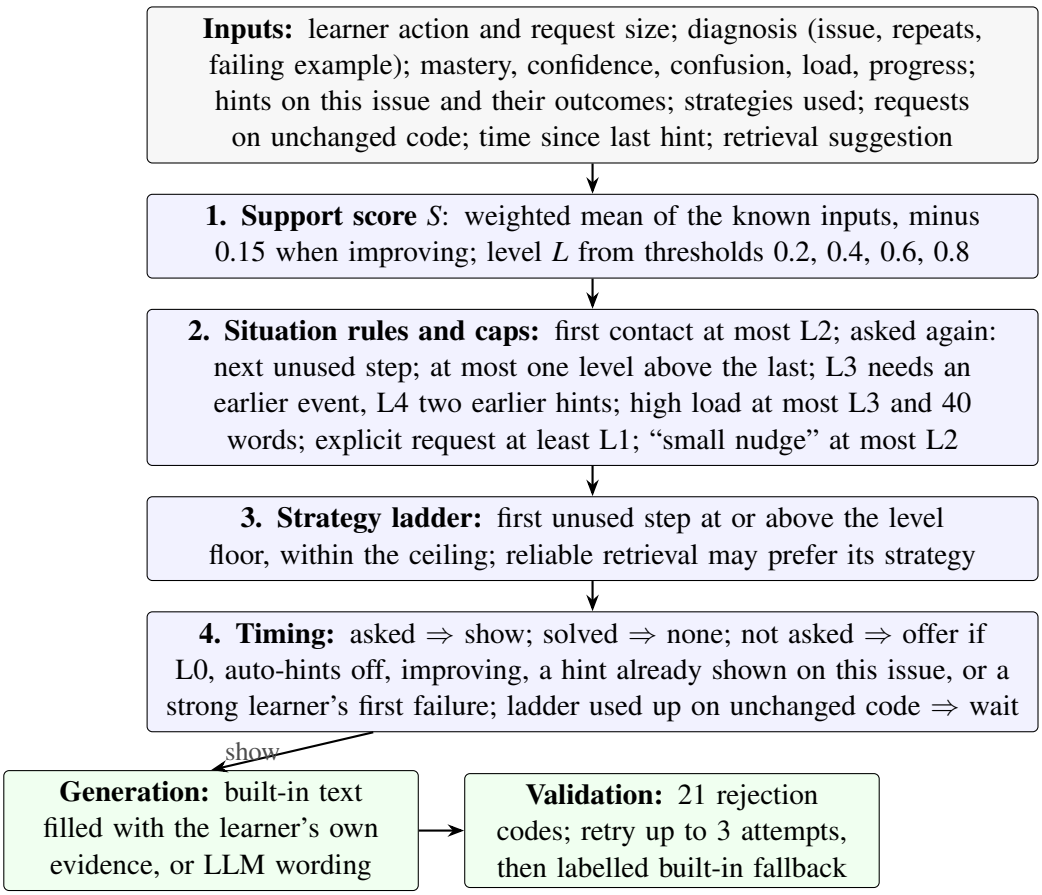


Figure 5.3: Decision flow of the Dynamic Scaffolding Decision Algorithm

0.22; confusion (low 0.2, moderate 0.55, high 0.9), 0.18; repeated issue $(\text{repeats}-1)/3$, 0.16; hints not followed by progress $/2$, 0.12; requests on unchanged code $/3$, 0.12; low confidence $1-\kappa$, 0.08; load (as confusion), 0.08; stalled time (seconds since last action$/600$), 0.04. The level is L0 if $S < 0.20$, L1 if $S < 0.40$, L2 if $S < 0.60$, L3 if $S < 0.80$ and L4 otherwise.

Help levels and ladders. Table 5.4 lists the levels and their presentation limits, and Table 5.5 the strategy ladders. Each issue climbs its own ladder one unused step at a time, which is what prevents help from repeating.

Table 5.4: Help levels and presentation limits

Level	Name	Format	Limit
L0	Independent attempt	No hint; an offer of help	–
L1	Conceptual question	A question	2 sentences, 45 words
L2	Targeted hint	Points at what to examine	3 sentences, 70 words
L3	Step-by-step outline	Numbered steps, no code	6 sentences, 110 words
L4	Analogous worked example	Code on a *different* task	7 sentences, 150 words

Table 5.5: Strategy ladders (level in parentheses)

Step	Most issues	Syntax errors	Missing names
1	conceptual question (1)	syntax pinpoint (2)	conceptual question (1)
2	targeted hint (2)	read the tokens (2)	targeted hint (2)
3	trace the failing case (2)	syntax checklist (3)	name options (2)
4	print debugging (2)	analogous line (4)	step outline (3)
5	step outline (3)	exact fix (4)	analogous example (4)
6	analogous example (4)	–	direct pointer (4)
7	direct pointer (4)	–	–

Algorithm 1 gives the timing decision that follows the choice of level and strategy. Its key property is that, after a first hint on an issue, help that the learner did not ask for is offered and not shown; help therefore never climbs the ladder unless the learner asks.

Worked example. A learner's second wrong submission with the same off-by-one bug, with mastery 0.25 (evidence present), moderate confusion, the issue seen in two attempts and one earlier hint not followed by progress; confidence, load and time are unknown and left out. The known weights sum to 0.80 and the score components are $0.22 \cdot 0.75/0.80 = 0.206$, $0.18 \cdot 0.55/0.80 = 0.124$, $0.16 \cdot 0.33/0.80 = 0.067$ and $0.12 \cdot 0.50/0.80 = 0.075$, giving $S = 0.472$ and level L2. The next unused step at or above L2 is `targeted_hint`. Because this was a submission, not a request, and a hint was already shown on the issue, the decision is to *offer* it (reason codes `SUPPORT_NEEDED`, `ESCALATE_NO_PROGRESS`, `OFFER_NEXT_STEP`). These values were recomputed by running the implemented functions and are also checked by `test_worked_example_in_the_documentation_is_accurate`.

Algorithm 1 DSDA timing decision

if solved and no question asked **then return** NONE (SOLVED)
end if
if service failure or no tests **then return** WAIT
end if
choose level and strategy (score, rules, ladder)
if no unused step and code unchanged since last hint **then return** WAIT (LADDER_EXHAUSTED)
end if
if learner asked (hint, "more", "nudge" or question) **then return** SHOW
end if
if level is L0 and not a syntax error **then return** OFFER
end if
if automatic hints switched off **then return** OFFER (AUTO_HINTS_OFF)
end if
if learner is improving and not a syntax error **then return** OFFER (PROGRESS_NO_HELP)
end if
if same issue and at least one hint already shown on it **then return** OFFER (OFFER_NEXT_STEP)
end if
if first failure, mastery ≥ 0.7 with evidence **then return** OFFER (OFFER_INDEPENDENT)
end if
return SHOW

Generation. Built-in text is chosen by strategy and filled only with observed evidence. For ten common logic and timeout issues, the hint quotes the learner's own failing call, the values returned and expected, the loop's line and source and its variable; other builders produce trace instructions, exactly what to print and where, analogous examples on a different task, and direct pointers in words. Text is trimmed to the level's sentence and word limits without cutting inside quoted code. When an LLM is configured, its prompt contains the task, the learner's code (marked as data), the evidence, the limits and every earlier hint, but never the reference solution or hidden tests.

Validation. Before anything is shown, the validator rejects a draft that: contains a code block or code-like lines below L4, or defines the function the learner must write; overlaps the reference solution; rewrites the learner's own code; reveals hidden-test values or mentions internal details; is too long or, at L1, is not a question; repeats an earlier hint; is vague with nothing specific; names a line that does not exist or test counts that match neither result; contradicts the diagnosis; ignores the diagnosed cause or only repeats Python's error message; or, for LLM text, names nothing concrete from the learner's code. Two of the checks are quantitative. Solution overlap uses token 6-grams:

$$\text{overlap}(h,\sigma) = \frac{|N_6(\sigma) \cap N_6(h)|}{|N_6(\sigma)|} \geq 0.25 \;\Rightarrow\; \text{reject}, \tag{5.7}$$

where $N_6(x)$ is the set of six-token sequences of text x and σ the reference solution. Repeti-

tion uses

$$\mathrm{rep}(a,b) = \max\big(\mathrm{SeqMatch}(a,b),\, J(a,b)\big) \geq 0.8 \;\Rightarrow\; \text{reject}, \tag{5.8}$$

where SeqMatch is the in-order word match ratio of Python's `difflib.SequenceMatcher` and J the Jaccard overlap of the word sets. A rejected draft is regenerated, each time told what was wrong; after three attempts a labelled built-in hint is used; if that would repeat, the next unused step is tried; if nothing passes, nothing is shown.

Outcome of a hint. On the learner's next checked attempt with changed code, each pending hint is marked *solved*, *progress* (more tests pass on the same kind of check, or the issue changed without getting worse) or *no progress*. These outcomes feed DSDA's "hints not followed by progress", the confusion estimate and, for consenting real accounts, the anonymous retrieval pool.

5.2.7 Agent 5: Reflection

Purpose. Ask one short question that makes the learner explain their own reasoning, when it helps, and reply to the answer like a tutor: no grading, no code and no fix.

When it asks. It defers when load is high, and skips when a question is still unanswered, when there is no learning event, for a syntax slip on unsolved code, before three failed attempts, within three events of the last question, when the question would repeat the hint, or when it was already asked. After a solve it asks about the technique, with three variants: solved with help, solved on the first try, and other.

The question is built from the cycle's evidence. For the example above it reads: "Trace `sum_to_n(3)` by hand: which values does `i` take in `for i in range(1, n):`, and which values does the answer need?" Anything not observed is left out rather than shown as a placeholder.

The reply. Algorithm 2 classifies the answer using regular expressions for the key ideas of the issue or technique (for example "where `range` stops" or "keeping a running total"), for reasoning words (because, so, since, therefore) and for honest uncertainty ("I don't know", "not sure"), and checks whether it is concrete (backticks, numbers, or the learner's own variable names).

Effects on other agents. The question, answer, reply and quality are stored. In the next cycle an unsure answer counts as one confusion signal. After a solve, an unsure answer holds back a plan that was moving on: the planner suggests one more problem of the same technique first. If the plan shown with the question already gives one more problem of that technique, it stands, so the learner is not told the same thing twice. Clear, partial and short answers leave the plan unchanged.

Algorithm 2 Reflection reply classification

$M \leftarrow$ key ideas matched; $R \leftarrow$ reasoning words present; $C \leftarrow$ answer is concrete
if uncertainty expressed and not R and not (C and $M \neq \emptyset$) **then return** UNSURE: kind reply and one small thing to try
end if
if fewer than 3 words and $M = \emptyset$ **then return** BRIEF: ask for a little more
end if
if $M \neq \emptyset$ and (C or R) **then return** SPECIFIC: name what was right, push one step further
end if
if $M \neq \emptyset$ or at least 8 words **then return** PARTIAL: one concrete thing to check
end if
return BRIEF

5.2.8 Agent 6: Learning Planner

Purpose. Recommend what to do next, with a reason, moving the learner from where they are towards harder work step by step.

Inputs. Topic mastery and evidence, prerequisites, each topic's techniques in lesson order, each problem's technique, difficulty and status, including whether it was solved fluently (first graded submission accepted with no hint shown before it), the current problem, attempts, the highest hint level, and the learner's goal and experience.

Outputs. An action, a title, a reason, reason codes, a problem and topic, whether the current task stays active, alternatives and a *route*: the current topic and up to three topics ahead in course order.

The next topic is the next one forward in the learner's track order; an earlier topic comes back only when it has work in it and is not yet secure. A weak prerequisite holds a topic back only while it still has problems to practise, and a held-back topic also holds back every topic that builds on it, so the planner never jumps past it. The route is a preview in course order and always shows the topic after the current one. The home screen uses the same rules as the plan after a submission, so the two always suggest the same next step.

5.2.9 Persistence and data model

The schema has 25 tables, created and evolved through five Alembic migrations. Figure 5.4 shows the core entities used by the learning cycle.

Every learning cycle writes one `interaction_records` row holding the outputs of all agents, so the next cycle can read what the tutor concluded and decided before. Hints store their validation result and the starting pass ratio, so their outcome can be judged later. `learner_topic_state` holds the current estimate per topic and `learner_state_snapshots` its history, which the profile page shows. Hidden-test contents, reference solutions and prompts are not stored in these records.

Algorithm 3 Next-step selection after an attempt

if problem not solved **then**
    if 4 or more attempts and a prerequisite topic is weak (mastery < 0.45 with evidence) **then return** practise prerequisite (task kept)
    end if
    return stay on this problem (`UNRESOLVED`)
end if
if solved with L3+ help, or with hints while over 60% of the topic's solves used hints **then return** same technique, same difficulty, on your own
end if
if topic secure: mastery ≥ 0.75, ≥ 2 solves without help, evidence ≥ 2.5 **then return** harder problem, else the next topic forward
end if
for level in easy, medium, hard **do**
    for technique in lesson order **do**
        if not practised at this level (easy: 2 solves or 1 fluent solve; above: 1 solve) and an unsolved problem exists **then**
            return that problem (`CONSOLIDATE_TECHNIQUE`, `NEXT_TECHNIQUE` or `STEP_UP_DIFFICULTY`)
        end if
    end for
end for
return next topic forward (`UNIT_EXHAUSTED`)

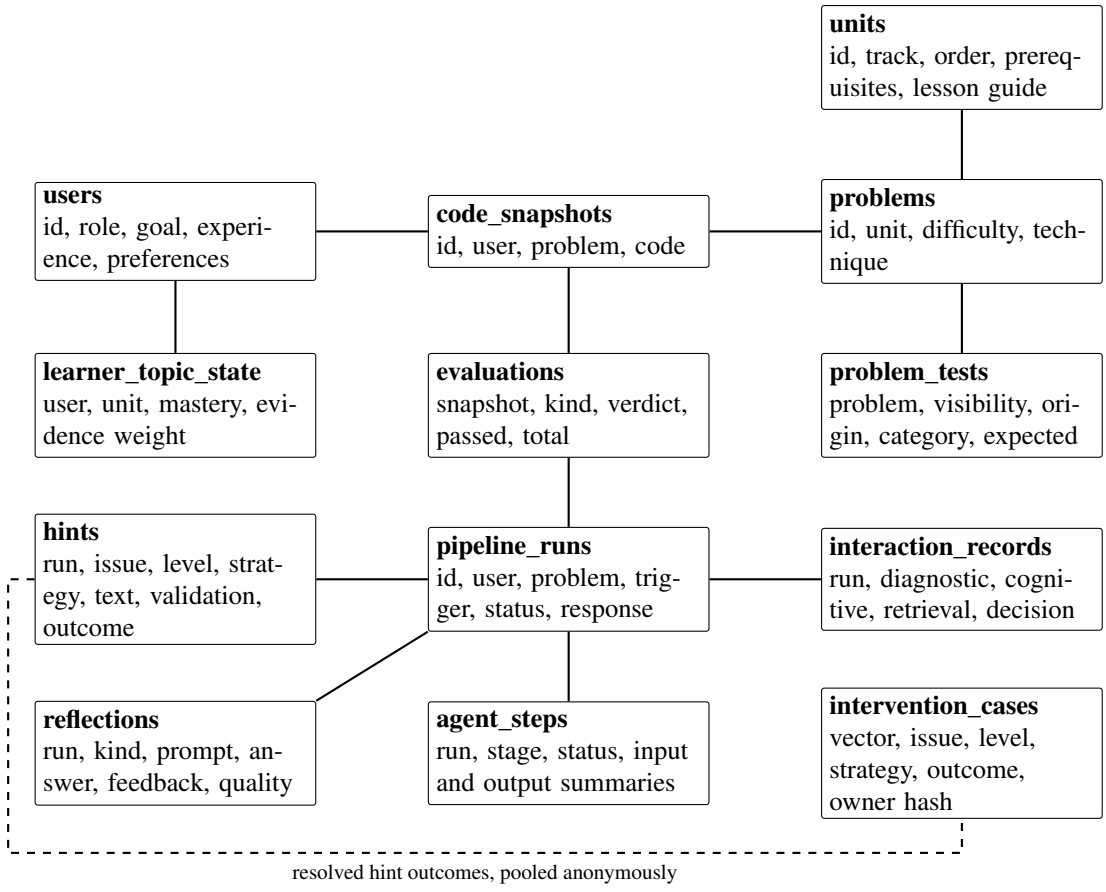


Figure 5.4: Core entities of the data model (simplified; lines show references)

5.2.10 Learner and teacher interfaces

The workspace places the problem, lesson, submissions and tutor in tabs beside the editor and the test results (Figure 6.3). Run, Submit and Ask for a hint are separate buttons, and a hint is never graded. The tutor pane shows the current hint with its level, its concept and a link to the matching lesson section, a collapsible "Why this hint?" explanation, "I need more help", a box for typed questions, the reflection card and the suggested next step with the route.

The teacher dashboard computes class measures from stored data: completion rate (problems solved divided by problems with at least one graded attempt), hints per solved problem, median active time to the first accepted submission, satisfaction (median of four Likert items) and learning gain $\frac{\text{post}-\text{pre}}{\text{max}-\text{pre}}$, which is shown only for learners who have taken both knowledge checks. It lists learners who may need support, with the evidence behind each flag.

Chapter 6

Results and Analysis

6.1 Evaluation setup

The evaluation in this phase verifies that the system is correct, safe and behaves as designed. It does not measure learning gains, because no study with real learners has been carried out yet; the planned study is described in Chapter 7.

Environment. All runs used the development machine described in Chapter 4 (Apple M2, 8 GB, macOS 27.2, Docker Desktop 29.8, Python 3.12.13, Node.js 26.4.0) with PostgreSQL 16.15 in a container. Every test used the real database, the real execution service with Docker sandbox containers, the real LangGraph pipeline and ChromaDB. No LLM was configured (`LLM_PROVIDER=none`), so every hint, question and reply used the built-in, evidence-based wording; the LLM path was tested with a scripted stand-in model.

Procedure.

- **Automated test suites.** The backend suite (pytest), the frontend unit suite (Vitest), the browser end-to-end suite (Playwright with Google Chrome and axe-core) and the execution-service suite were run against the code. For browser tests, a separate API and web server were started on other ports with raised rate limits, so that test traffic did not affect the developers' server.
- **Agent audit.** The script `scripts/agent_audit.py` registers a fresh learner for each of nine scenarios, submits code or asks for hints on the problem "Sum from 1 to N", and reads each agent's output back from the run response and its per-stage trace. It checks 37 behavioural rules plus two rules over every hint shown.
- **Curriculum verification.** Every problem's reference solution was run against all of its tests in the sandbox, and every starter program was confirmed to fail.
- **Interface inspection.** Screenshots were captured by a script that drives the running application as a demo learner and a demo teacher.

Safeguards against invalid results.

- The suites were run one at a time, because running them in parallel on the 8 GB machine overloaded Docker and produced false failures in an earlier run.
- The final runs were made after the last code change; counts reported here are from those runs.
- The audit reads outputs from the stored traces rather than from the code under test, so it checks what the agents actually produced.
- All synthetic data (the teacher demo class and demonstration retrieval records) is labelled in the interface and is not used as evidence that the tutor works.

6.2 Evaluation criteria

Table 6.1 defines the criteria used.

Table 6.1: Evaluation criteria

Criterion	**Measure and interpretation**	**Target**
Functional correctness	Tests passed ÷ tests run, per suite.	100% on the final code.
Agent behaviour	Audit checks passed ÷ checks, over nine scenarios that exercise all six agents.	All checks pass.
Non-repetition of help	Distinct hints among repeated requests on unchanged code; highest pairwise similarity $\mathrm{rep}(a,b)$ (Section 5.2).	All distinct; similarity < 0.8.
Controlled escalation	Level change between consecutive hints on one issue; hints shown without a request after a first hint.	At most $+1$; none shown unasked.
Hint specificity	Share of hints that name a line, value or name from the learner's code, and share free of generic advice.	100% of hints shown.
Solution protection	Responses containing hidden-test values or reference solutions; hints exceeding the solution-overlap threshold.	None.
Curriculum validity	Problems whose reference solution passes all tests and whose starter fails.	All 496 problems.
Test coverage	Tests per problem; share of problems with at least 30 tests.	Coverage reported.
Accessibility	Automated WCAG 2.0 and 2.1 A and AA violations on main screens, both themes.	Zero violations.

6.3 Result

6.3.1 Automated test suites

Table 6.2 summarises the final runs. Every suite passed in full.

Table 6.2: Results of the automated test suites on the final code

Suite	Result	What it exercises
Backend (pytest)	551 of 551	API, all six agents, the full pipeline, persistence, retrieval, policy, hint quality, imports and content, with real PostgreSQL, sandbox, LangGraph and ChromaDB.
Browser end-to-end (Playwright)	59 of 59	54 learner and teacher flows in real Chrome, and 5 automated accessibility tests in light and dark themes.
Frontend unit (Vitest)	21 of 21	Shared interface components, draft restoration, test-case editing, display formatting, saved-data migration and new-tab links.
Execution service	39 of 39	26 execution tests and 13 isolation tests in real containers (run on 9 November 2026; this service did not change afterwards).
Agent audit	37 of 37, plus 2 of 2	Nine scenarios through the running stack; two further checks over all 17 hints shown.
Curriculum verification	496 of 496	Each reference solution passes all of its tests (15,717 in total) and each starter fails, in the sandbox.
Static checks	Passed	Ruff lint (backend), TypeScript strict type check and production build (frontend).

Figure 6.1 shows how the 551 backend tests are distributed across test files. The largest file covers the repeated-hint behaviour, the area where the most serious defect was found during development (Section 6.4).

6.3.2 Agent audit

All 37 behavioural checks and both checks over every hint passed in the final audit run (Figure 6.2). Table 6.3 lists representative checks with the evidence recorded by the audit.

Table 6.3: Representative audit checks and recorded evidence

Check	Evidence recorded in the final run
Every request on unchanged code gets a different hint	4 requests, 4 different hints; pairwise similarities 0.16 to 0.38 (all below 0.8).
Escalation is gradual	Levels 2, 2, 2, 3; strategies targeted hint, trace the failing case, print debugging, step outline.

Check	**Evidence recorded in the final run**
Retrieval leaves out approaches already shown	Excluded strategies grew with each request: none; targeted hint; then also trace; then also print debugging.
Syntax error is diagnosed with its line	`SyntaxError: expected ':'`; first hint: "Look at line 1. A line that opens a block … must end with one particular punctuation mark. Which mark is missing there?"
Runtime cause is told apart from a generic error	`UnboundLocalError` diagnosed as "used before assignment", confidence 0.95.
Endless loop is diagnosed as a timeout	`timeout:loop_no_progress`; the hint asks the learner to follow `while i <= n:` on line 4 by hand.
Help is offered, not pushed	First failed submission: a hint. Next two failed submissions: `offer_help` both times. Asking then: next step, level 2.
Overloaded learner	One short action of 33 words and no reflection question (deferred).
Unknown stays unknown	With no history, confusion is reported as unknown rather than assumed.
Reflection holds back a plan that was moving on	After a first-try solve the plan was "Count the Digits" (next technique); the answer "not sure, I just changed it until it passed" changed it to "Sum Odds Only" (one more running-totals problem).
Reflection does not repeat the plan	After a solve with help the plan already gave one more running-totals problem; the same unsure answer left it unchanged.
Planner moves forward	Route: Loops, Functions, Strings, Lists and dictionaries; the home screen suggested the same next problem as the plan.
Every hint is specific	All 17 hints shown named a line, value or name from the learner's code, and none used generic advice such as "check your logic".

6.3.3 Tutoring behaviour in the interface

Figures 6.3 to 6.7 show one learner session on "Sum from 1 to N", captured from the running application. The code loops over `range(1, n)` and so misses the last value.

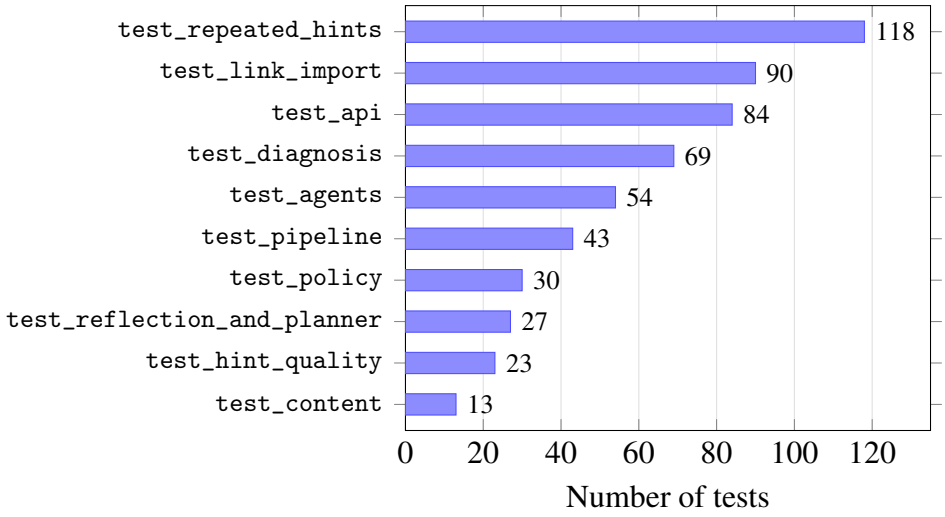


Figure 6.1: Backend tests per test file (551 in total, all passed)

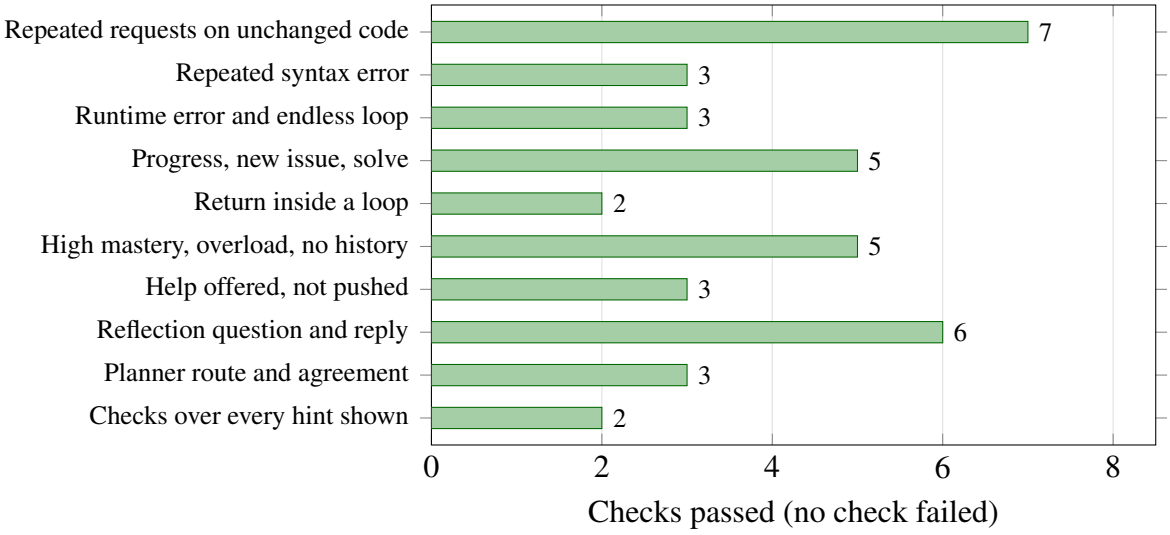


Figure 6.2: Agent audit: checks passed per scenario (39 of 39)

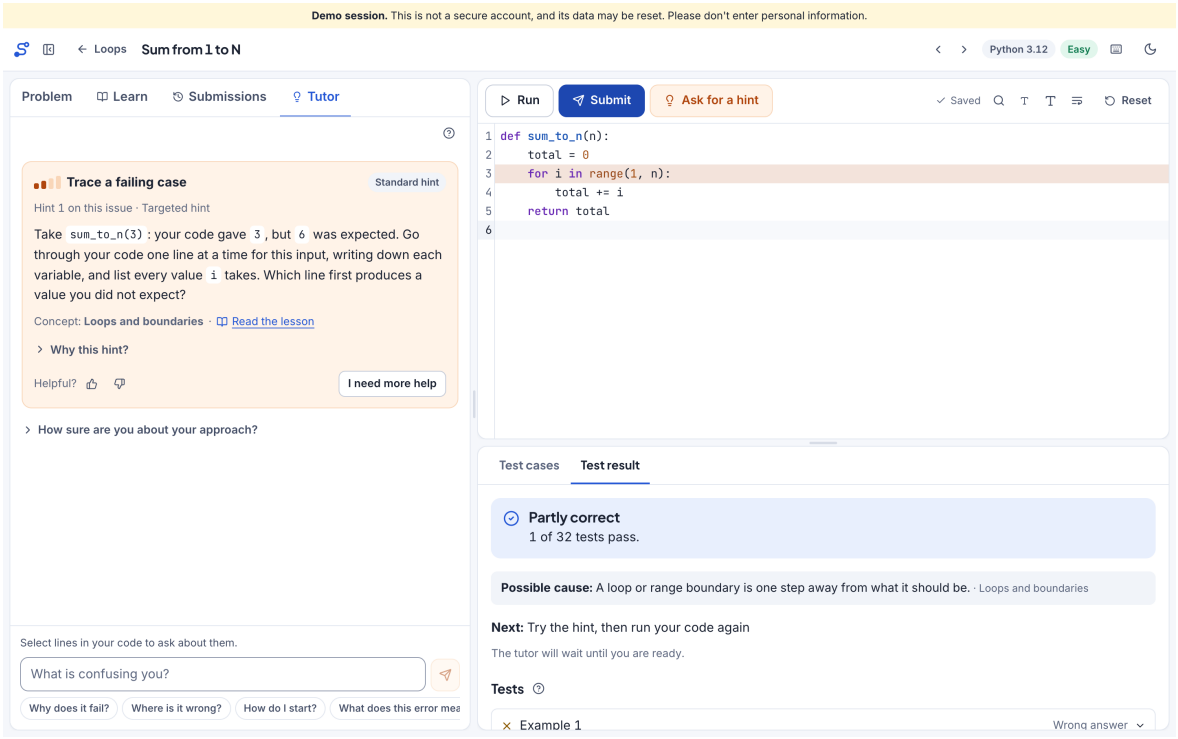


Figure 6.3: First wrong submission: the tutor gives one targeted hint built from the learner's own failing call, values and loop variable, with the concept, a link to the lesson and a "Why this hint?" explanation

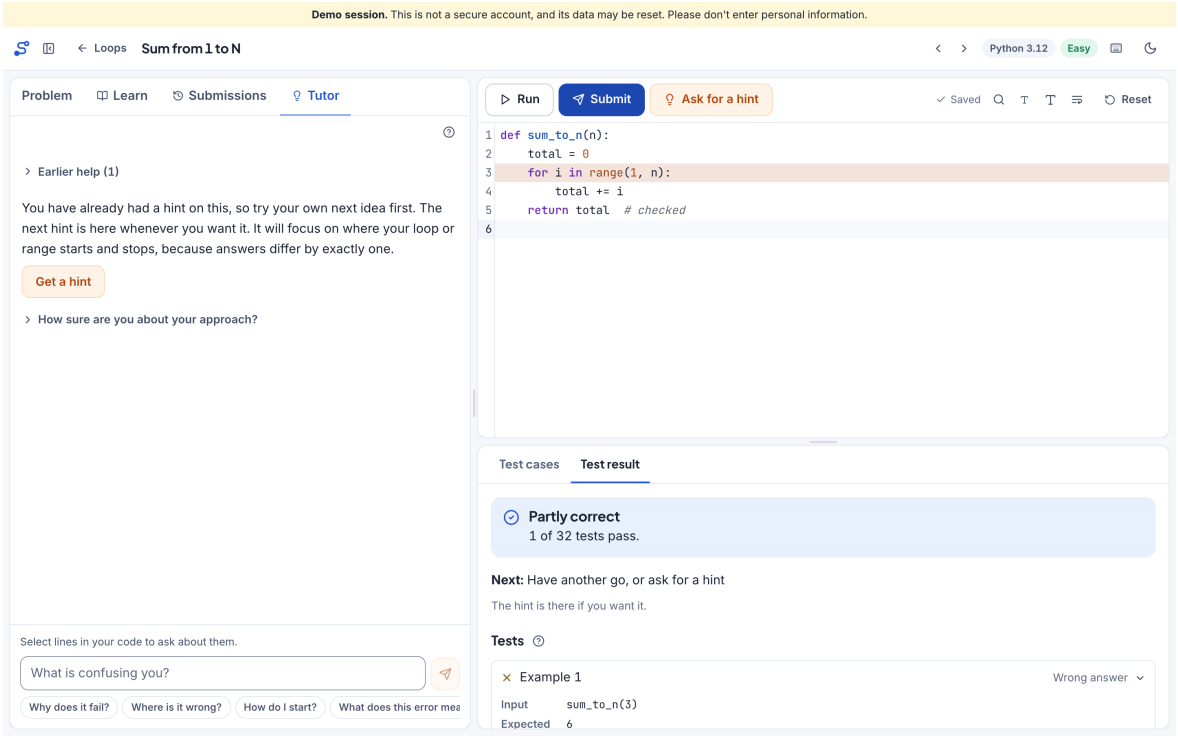


Figure 6.4: Second wrong submission on the same issue: the next hint is offered, not shown, and says once what it will focus on

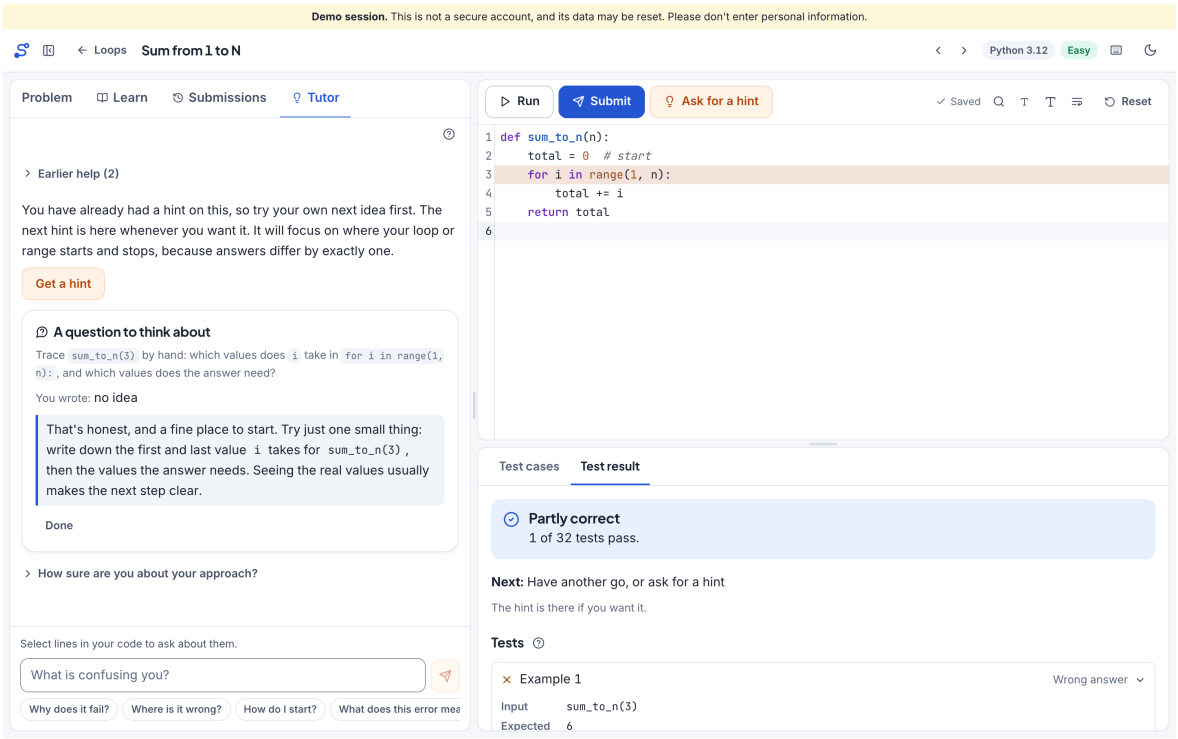


Figure 6.5: After three failed attempts: a reflection question about the learner's own loop, and the tutor's reply to the answer "no idea"

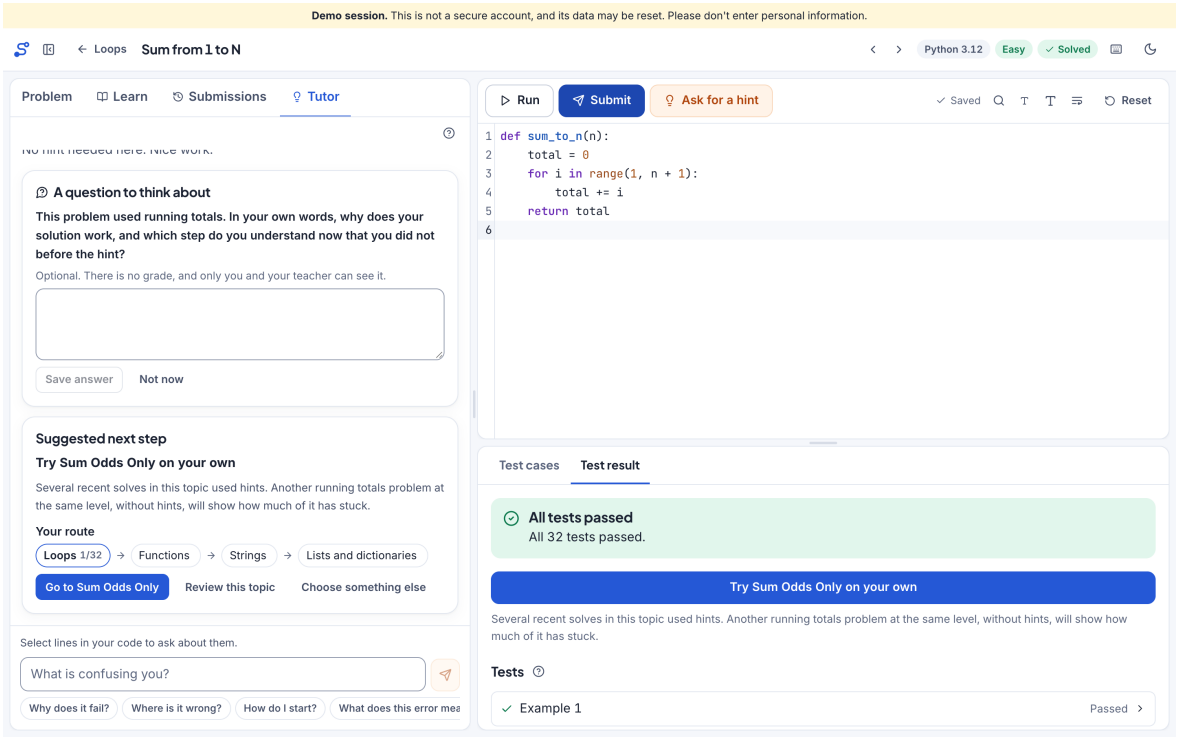


Figure 6.6: After the solve: a question about the technique, and a next step on the same technique without hints, with the route ahead

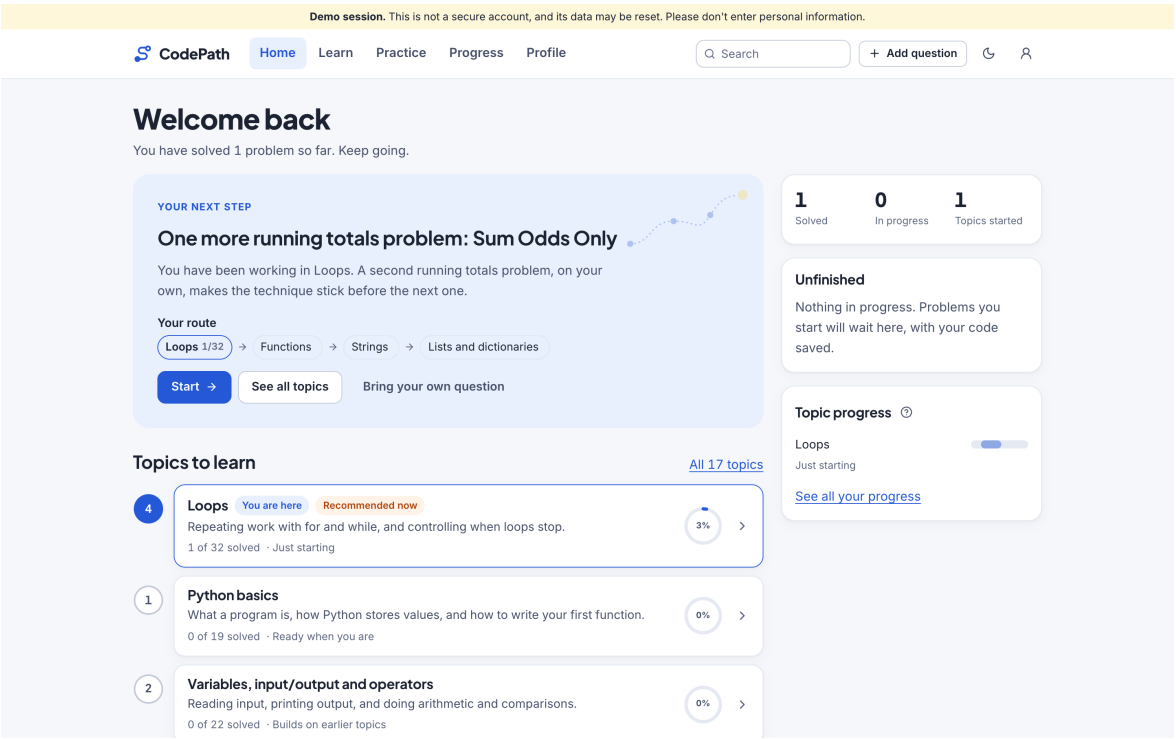


Figure 6.7: Home screen after the session: the same next step as the plan, with the route through the course

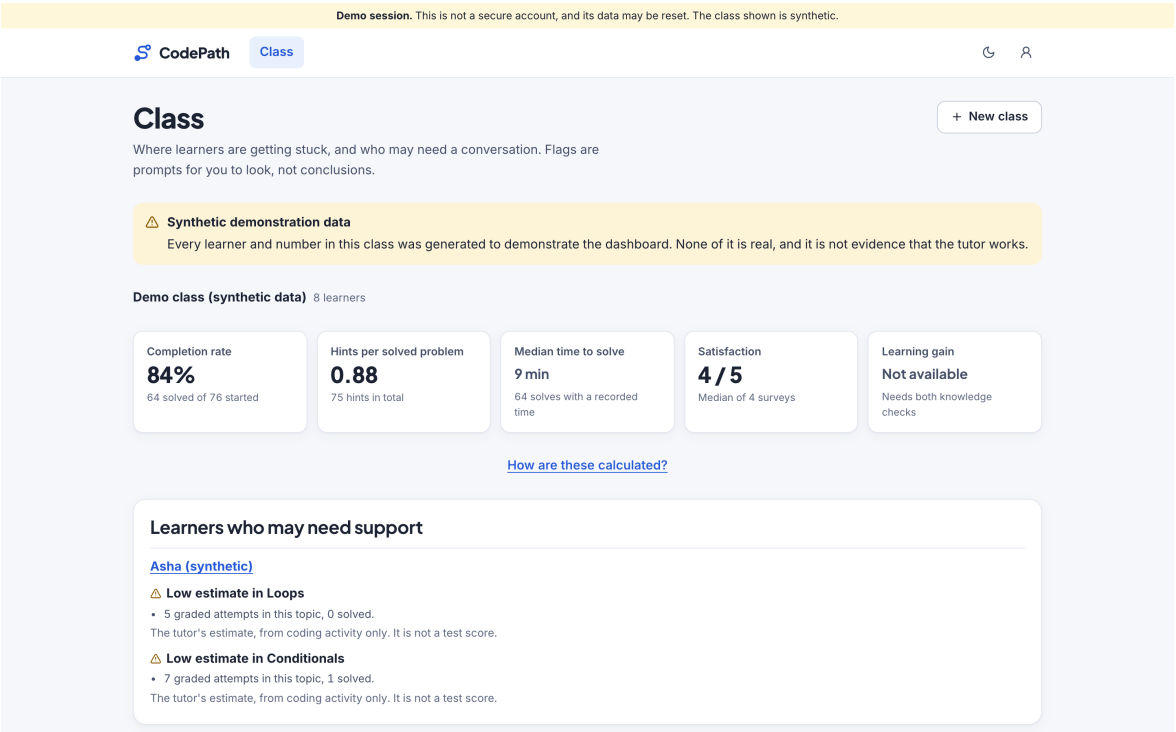


Figure 6.8: Teacher dashboard with the clearly labelled synthetic demonstration class; the numbers are generated and are not evidence that the tutor works

6.3.4 Learner-state estimates

Figure 6.9 shows how the implemented mastery update responds to three attempt sequences, computed by running the Cognitive State agent's own functions with its configured parameters. A learner who solves five times without help rises quickly; a learner who fails twice, solves once with a level-2 hint and then solves twice alone first drops and then recovers; a learner who keeps failing stays low. The dotted lines mark the band boundaries, and the band stays "not enough evidence" until the total evidence weight reaches 1.5. These curves illustrate the model's behaviour; they are not data from real learners.

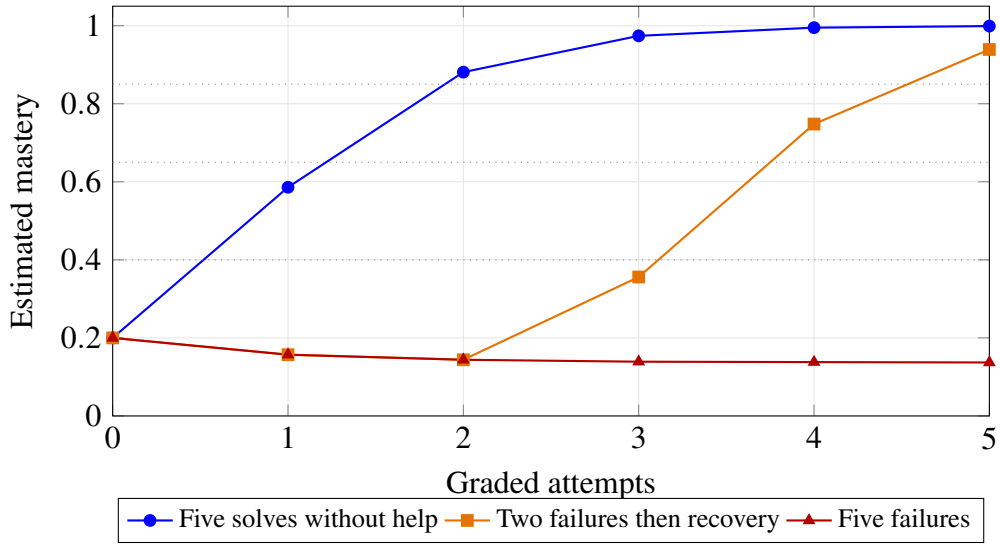


Figure 6.9: Mastery estimates produced by the implemented weighted knowledge-tracing update for three attempt sequences

6.3.5 Curriculum and tests

The curriculum contains 496 verified problems in 17 topics (Figure 6.10). Earlier topics are mostly easy and later data-structure topics mostly medium and hard. Of the 15,717 tests, 1,000 are visible and 14,717 hidden; 487 problems have at least 30 tests (median 32, maximum 34). The nine problems with fewer tests have few possible inputs, for example a problem that takes no input. Hidden tests are chosen to cover different kinds of input (Figure 6.11).

6.4 Comparative Analysis

6.4.1 Before and after the agent improvements

During development, a reproduction of the repeated-hint defect recorded the behaviour before the fix, with five requests on unchanged code in each scenario. Table 6.4 compares it with the final system.

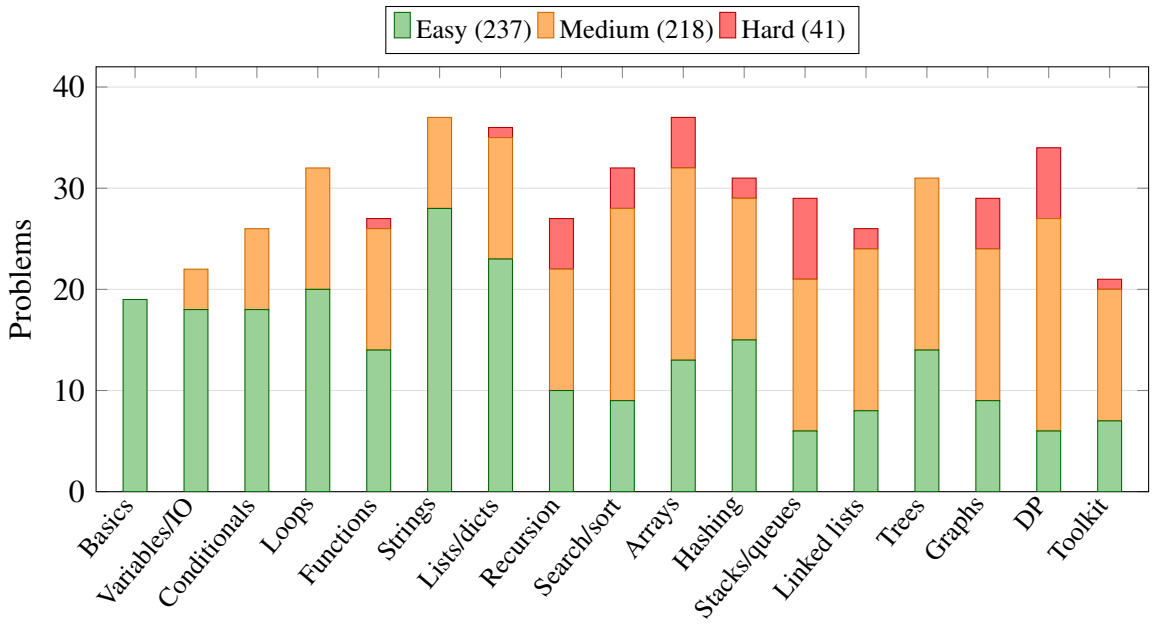


Figure 6.10: Curriculum problems per topic and difficulty (496 in total)

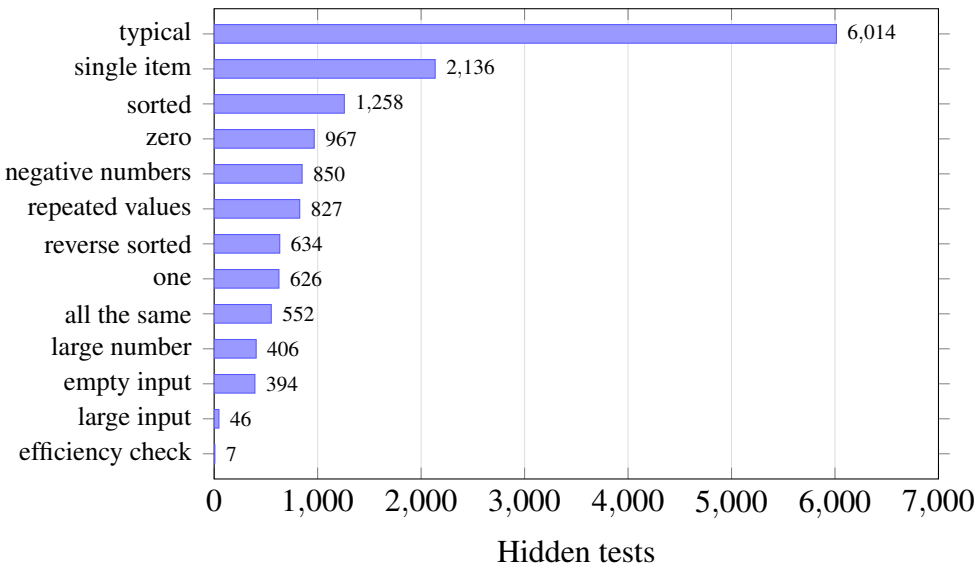


Figure 6.11: Hidden tests by kind of input (14,717 in total)

Table 6.4: Behaviour before and after the agent improvements

Behaviour	**Before**	**After (final system)**
Repeated hint requests on unchanged code	1 distinct hint in 5 requests in every scenario (syntax, missing colon, off-by-one, runtime, timeout); constant level and support score 0.208	4 different hints in 4 requests, highest similarity 0.38; levels 2, 2, 2, 3; a different strategy each time; then the tutor waits for a code change
Wrong submissions without asking	Each further failed submission on an issue could show the next, more revealing hint automatically	After a first hint, help is offered and shown only when asked, at most one level higher
Reflection	Fixed questions, the same for every learner; answers stored with no reply and no effect	Questions name the learner's own code or technique; every answer gets a reply; an unsure answer reaches the Cognitive State and the Planner
Planning	Next problem chosen by difficulty only; next topic could be an earlier topic with no work in it	Technique-by-technique sequencing at each difficulty; forward topic order; route shown; home screen and plan agree
Hints for logic mistakes	8 of 16 hints in the first audit run named nothing concrete from the learner's code	All 17 hints in the final run name a line, value or name from the learner's code

6.4.2 Comparison with existing approaches

Table 6.5 compares CodePath's implemented capabilities with those described for the most closely related reviewed systems. The entries for other systems state only what their published records describe; "Not reported" does not mean the system lacks the capability. The CodePath row describes implemented and tested functions, not measured learning effects.

Table 6.5: Capabilities of CodePath and closely related systems

System	**Persistent learner model**	**Decides how much help**	**Withholds full solutions**	**Retrieval**	**Several agents**	**Live programming practice**
Knowledge tracing [1]	Yes (per skill)	Selects exercises until mastery	Not applicable	Not reported	Not reported	Yes
CodeHelp [6]	Not reported	Not reported	Yes (guardrails)	Not reported	Not reported	Yes
CodeAid [7]	Not reported	Not reported	Yes	Not reported	Not reported	Yes
PyTutor [8]	Not reported	Structured hint types	No (hints include coding solutions)	Not reported	Not reported	Yes
RAGMan [9]	Not reported	Not reported	Not reported	Course material	Not reported	Yes
Ruffle&Riley [10]	Not reported	Not reported	Not applicable	Not reported	Yes (two agents)	No (biology)
TRAVER [12]	Yes (knowledge tracing)	Not reported	Not reported	Not reported	Not reported	Simulated students
CodePath (this work)	**Yes: mastery with range, confusion, load, progress**	**Yes: DSDA level, strategy and timing**	**Yes: validator and tests**	**Similar learners' outcomes**	**Yes: six agents**	**Yes**

6.4.3 Comparison with the proposal's evaluation metrics

The initial submission proposed four evaluation measures: learning gain, hint efficiency and hints per problem, task completion rate and time to completion, and student satisfaction. All four are implemented as defined formulas in the teacher dashboard (Section 5.2), and their computation is covered by tests. Their values for real learners are not yet available: the only numbers currently shown come from the synthetic demonstration class (Figure 6.8) and are labelled as such. Learning gain is deliberately shown only for learners who have taken both knowledge checks.

6.5 Discussion and Interpretation

Objective O1 (multiple collaborating agents) is met: every learner action runs through six agents with typed contracts, the agents read each other's outputs and the stored history, and pipeline tests and the audit confirm the order and the data passed between them.

Objective O2 (cognitive state) is met as a set of explainable behavioural estimates. The audit shows that confusion stays unknown without evidence, rises with repeated requests, and that progress is detected when a new version gets further. These estimates have not been compared with any external measure of a learner's state, so they should be read as the tutor's working hypotheses.

Objective O3 (similar-learner retrieval) is met as a mechanism: retrieval filters by similarity, outcome, difficulty and topic, excludes approaches already seen and refuses to answer on thin evidence. Its real value depends on outcomes from real learners, which do not yet exist.

Objective O4 (scaffolding algorithm) is met. DSDA produces a level, strategy and timing with stated reasons, its documented worked example is reproduced exactly by the code, and the audit confirms the key teaching behaviours: help does not repeat, rises at most one level at a time, is not pushed after a first hint, stays short for an overloaded learner and never contains a solution.

Objective O5 (effectiveness on datasets) is partly met. The curriculum dataset is verified, and the system's behaviour is verified across realistic scenarios. Whether the tutor improves learning is not yet known.

The results also support the project's central claim about *how* help is given. The comparison in Table 6.4 shows that the final system behaves much more like a patient human tutor than the earlier version: it waits for the learner to try their own idea, gives one specific next step when asked, reacts to what the learner says in reflection, and moves forward through the course at a pace set by the learner's own work. The literature gives reasons for caution: help-seeking feedback can change behaviour without improving learning [4], and engaging

tutors do not necessarily outperform reading [10]. This is why the next phase must measure learning directly.

6.6 Limitations

- **No evidence yet of learning improvement.** The system has not been used by real learners. Every weight and threshold in the policy is a starting value chosen by judgement.
- **Estimates are behavioural.** Confusion and load are inferred from behaviour and have not been validated against an external measure.
- **Retrieval data.** Similar-situation records are currently synthetic demonstration cases plus outcomes pooled from automated test and audit runs; the latter count as "live" in the development database, so a fresh database is needed for any study.
- **Real model untested.** The available model key had no credit, so model-written wording, rejection rates and latency are unknown; all observed behaviour used built-in wording.
- **Heuristic checks.** The validator detects copied and lightly reworded solutions and repeats, but a determined rewording could pass. Reflection replies spot key ideas by keywords, so an unusual but correct answer can be read as partial.
- **Rule coverage.** Error-cause analysis covers common causes; an unusual error falls back to general help with lower confidence. Built-in hints quote the learner's own code for ten common logic and timeout issues.
- **Execution and deployment.** The sandbox was tested on Docker Desktop for macOS, not on a Linux server, and has not been independently audited. The worker pool, queue and rate limits are held in memory, so the API runs as a single process.
- **Content and data.** Only Python is supported. The curriculum was written for this project and its difficulty labels are the authors' judgement. CodeNet was not imported.
- **Testing scope.** Accessibility was checked automatically, not with screen-reader users, and browser tests ran on Chrome only.

Chapter 7

Conclusion

7.1 Conclusion

This project addressed a specific problem in programming education: AI assistants can answer a beginner's question instantly, but help that is too much, too soon or the same for everyone can take away the thinking a learner needs to do. The project designed and implemented CodePath, a web-based tutor for novice Python programmers based on the Cognitive-State Guided Multi-Agent Adaptive Scaffolding Framework.

The implemented contribution consists of:

- a cyclic LangGraph workflow of six agents with typed contracts, shared persistent history and a pause after every cycle, so that the tutor acts only in response to the learner;
- a rule-based Diagnostic agent that uses test results and syntax-tree analysis to label issues and quote the learner's own code;
- a Cognitive State agent that applies an evidence-weighted knowledge-tracing update and reports confusion, load and progress with their evidence, or "unknown";
- a Similar Learner Retrieval agent that combines ChromaDB cosine search with outcome records in PostgreSQL and refuses to answer on thin evidence;
- the Dynamic Scaffolding Decision Algorithm, which decides whether, when and how much to help through an explainable score, strategy ladders and timing rules, followed by validated generation that never reveals a solution and never repeats;
- a Reflection agent that asks about the learner's own reasoning and replies to the answer, and a Learning Planner that sequences practice technique by technique and moves forward through the course;
- a safe execution service, a verified curriculum of 496 problems with 15,717 tests, and learner and teacher interfaces.

The evidence supports objectives O1 to O4 as software. On the final code, 551 backend tests, 59 browser tests, 21 unit tests and 39 execution-service tests passed, every curriculum problem was verified in the sandbox, and an audit of nine scenarios passed all 37 of its behavioural checks and both checks over every hint. Objective O5 is partly achieved: the system's behaviour and its curriculum are verified, but its effect on learning has not yet been measured with real learners. Consistent with Chapter 6, this report therefore claims a working, tested tutor whose teaching behaviour matches its design, not a demonstrated improvement in learning.

7.2 Future work

- **Learner study (next phase, highest priority).** A pilot with about 30 or more learners, randomly assigned and balanced on the starting knowledge check, comparing CodePath with the same interface using a plain LLM assistant or fixed hints. Measures: normalised learning gain between parallel pre- and post-checks with an effect size, completion rate, hints per solved problem, time to completion and satisfaction. Ablations: without retrieval, with DSDA constrained through `policy.yaml`, and with a fixed problem order. *Dependencies:* institutional ethics approval, informed consent, participants and a fresh database. *Validation:* pre-registered analysis and treating a small sample as preliminary.
- **Evaluation with a real model.** Configure an LLM provider and measure the rate at which drafts are rejected, leak rates against a fixed set of solution-extraction prompts, wording quality and latency. *Dependency:* model access with credit.
- **Learning the policy parameters.** Fit the knowledge-tracing parameters and DSDA weights to logged interactions, and compare the fitted model's prediction of the next result with the current values, or with Deep Knowledge Tracing [2], using AUC. *Dependency:* data from the learner study.
- **Offline evaluation on CodeNet.** Import CodeNet [5] problem statistics and attempt sequences, check difficulty labels against acceptance rates, and replay sequences through the Cognitive State agent; simulated learners [11] could also be used to test policies before a study.
- **Deployment hardening.** Test the sandbox on a Linux server and arrange an independent security review; move the queue and rate limits to a shared store so the API can run as several processes; add email verification, password reset and automated data retention.
- **Broader scope.** Add JavaScript and then Java or C++ once Python is stable, extend the rule coverage of the Diagnostic agent, and test accessibility with screen-reader users.

References

[1] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Model. User-Adapt. Interact.*, vol. 4, no. 4, pp. 253–278, 1995, doi: 10.1007/BF01099821.

[2] C. Piech, J. Spencer, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, 2015. [Online]. Available: arXiv:1506.05908.

[3] V. Rus, N. B. Niraula, and R. Banjade, "DeepTutor: An effective, online intelligent tutoring system that promotes deep learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 29, no. 1, 2015, doi: 10.1609/aaai.v29i1.9269.

[4] V. Aleven, I. Roll, B. M. McLaren, and K. R. Koedinger, "Help helps, but only so much: Research on help seeking with intelligent tutoring systems," *Int. J. Artif. Intell. Educ.*, vol. 26, no. 1, pp. 205–223, 2016, doi: 10.1007/s40593-015-0089-1.

[5] R. Puri *et al.*, "Project CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," arXiv:2105.12655, 2021, doi: 10.48550/arXiv.2105.12655.

[6] M. Liffiton, B. Sheese, J. Savelka, and P. Denny, "CodeHelp: Using large language models with guardrails for scalable support in programming classes," arXiv:2308.06921, 2023, doi: 10.48550/arXiv.2308.06921.

[7] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, "CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs," in *Proc. CHI Conf. Human Factors Comput. Syst. (CHI '24)*, Honolulu, HI, USA, 2024, doi: 10.1145/3613904.3642773.

[8] A. C. M. Yang, J.-Y. Lin, C.-Y. Lin, and H. Ogata, "Enhancing Python learning with PyTutor: Efficacy of a ChatGPT-based intelligent tutoring system in programming education," *Comput. Educ. Artif. Intell.*, vol. 7, Art. no. 100309, 2024, doi: 10.1016/j.caeai.2024.100309.

[9] I. Ma, A. Krone Martins, and C. V. Lopes, "Integrating AI tutors in a programming course," accepted at SIGCSE Virtual 2024, arXiv:2407.15718, 2024, doi: 10.48550/arXiv.2407.15718.

[10] R. Schmucker, M. Xia, A. Azaria, and T. Mitchell, "Ruffle&Riley: Insights from designing and evaluating a large language model-based conversational tutoring system," arXiv:2404.17460, 2024, doi: 10.48550/arXiv.2404.17460.

[11] Y. Zhan, Q. Liu, W. Gao, Z. Zhang, T. Wang, S. Shen, J. Lu, and Z. Huang, "CoderAgent: Simulating student behavior for personalized programming learning with large language models," in *Proc. 34th Int. Joint Conf. Artif. Intell. (IJCAI-25)*, 2025, pp. 293–301, doi: 10.24963/ijcai.2025/34.

[12] J. Wang, Y. Dai, Y. Zhang, Z. Ma, W. Li, and J. Chai, "Training turn-by-turn verifiers for dialogue tutoring agents: The curious case of LLMs as your coding tutors," in *Findings Assoc. Comput. Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 12416–12436, doi: 10.18653/v1/2025.findings-acl.642.

[13] G. Pitts, A. P. Hridi, and A. B. Lekshmi Narayanan, "A survey of LLM-based applications in programming education: Balancing automation and human oversight," in *Proc. 4th Workshop Bridging Human-Comput. Interact. Natural Lang. Process. (HCI+NLP)*, Suzhou, China, 2025, pp. 255–262, doi: 10.18653/v1/2025.hcinlp-1.21.

[14] D. Wood, J. S. Bruner, and G. Ross, "The role of tutoring in problem solving," *J. Child Psychol. Psychiatry*, vol. 17, no. 2, pp. 89–100, 1976, doi: 10.1111/j.1469-7610.1976.tb00381.x.

[15] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cogn. Sci.*, vol. 12, no. 2, pp. 257–285, 1988, doi: 10.1207/s15516709cog1202_4.

[16] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educ. Psychol.*, vol. 46, no. 4, pp. 197–221, 2011, doi: 10.1080/00461520.2011.611369.

[17] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

[18] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[19] LangChain, "LangGraph," GitHub repository. [Online]. Available: `https://github.com/langchain-ai/langgraph`

[20] Chroma, "Chroma: the open-source embedding database," GitHub repository. [Online]. Available: `https://github.com/chroma-core/chroma`

[21] S. Ramírez, "FastAPI," GitHub repository. [Online]. Available: `https://github.com/fastapi/fastapi`

[22] Microsoft, "Playwright," GitHub repository. [Online]. Available: `https://github.c`

om/microsoft/playwright

[23] Deque Systems, "axe-core: Accessibility engine for automated web UI testing," GitHub repository. [Online]. Available: https://github.com/dequelabs/axe-core

Appendix A

Source Code and Reproducibility

The complete source code, curriculum, tests and documentation are in the project repository `https://github.com/lokesh12125/AdaptiveAITutor`, on the branch `Trellis`. Version used for this report: commit [commit identifier, to be recorded after the final work is committed]. The repository's `.env.example` lists every configuration value without secrets; no passwords, tokens or keys are included in this report or in the repository.

The repository is organised as follows:

- `backend/`: FastAPI application; `app/tutoring/` holds the six agents (`diagnostic.py`, `error_analysis.py`, `cognitive.py`, `retrieval.py`, `dsda.py`, `hints.py`, `reflection.py`, `planner.py`), the graph (`graph.py`, `nodes.py`) and the policy file `policy.yaml`; `migrations/` holds the five database migrations; `tests/` the backend tests.
- `runner/`: the execution service and the sandbox image definition.
- `frontend/`: the React application, unit tests and the Playwright browser tests (`e2e/`).
- `content/`: lessons, topic notes and technique guides; `scripts/`: the problem bank, test generation, content verification and the agent audit.
- `docs/`: architecture, agent contracts, DSDA, security, evaluation, limitations and test results; `report/`: this report, its figure data and the scripts that produced them.

A.1 Setup and execution

Prerequisites. Docker (running), the `uv` Python package manager, Python 3.12 and Node.js 20 or later. No AI key is needed.

Automatic setup. From the repository root:

```
./scripts/setup.sh
./scripts/dev.sh
```

`setup.sh` creates `.env` with fresh secrets, starts PostgreSQL on port 5544, builds the sandbox image, installs the dependencies, runs the migrations and loads the content. `dev.sh` starts the execution service (port 8102), the API (port 8101) and the web application (port 5173). Open `http://127.0.0.1:5173` and choose "Try as a learner" or "Try as a teacher".

Manual setup.

```
cp .env.example .env    # then set RUNNER_TOKEN
docker compose up -d db
docker build -t csmasf-sandbox:py312 runner/sandbox

cd runner && uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -e '.[dev]'
.venv/bin/uvicorn runner_service.main:app --host 127.0.0.1 --port 8102

cd backend && uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -e '.[dev]'
.venv/bin/alembic upgrade head
.venv/bin/python -m app.seed
.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8101

cd frontend && npm ci && npx vite --host 127.0.0.1 --port 5173
```

Running the tests (one suite at a time):

```
cd runner && .venv/bin/python -m pytest
cd backend && .venv/bin/python -m pytest
cd frontend && npm test
cd frontend && E2E_CHROME_CHANNEL=chrome npm run e2e
cd backend && PYTHONPATH=. .venv/bin/python ../scripts/verify_content.py
backend/.venv/bin/python scripts/agent_audit.py http://127.0.0.1:8101
```

The browser tests need the stack running with raised rate limits: set `RATE_RUN_PER_MINUTE`, `RATE_SUBMIT_PER_MINUTE`, `RATE_HINT_PER_MINUTE`, `RATE_AUTH_PER_MINUTE` and `RATE_IMPORT_PER_MINUTE` to a large value such as 100000 before starting it.

Reproducing the report's figures. The chart data is computed from the database and from the agents' own code, and the screenshots are captured from the running application:

```
backend/.venv/bin/python report/tools/make_figure_data.py
NODE_PATH=frontend/node_modules node report/tools/screenshots.cjs
cd report && tectonic -X compile main.tex
```

A.2 Supporting material

Test files. Table A.1 lists the backend test files and what each verifies.

Table A.1: Backend test files

File	Tests	What it verifies
`test_repeated_hints.py`	118	Persistent interaction state, varied help on repeated requests, retrieval filtering and privacy.
`test_link_import.py`	90	Adding questions from pasted text and links, site rules and labelled suggested tests.
`test_api.py`	84	Authentication, demo access, rate limits, progress, profile, teacher views and dashboard measures.
`test_diagnosis.py`	69	Diagnostic rules and error-cause analysis.
`test_agents.py`	54	Individual agents: cognitive estimates, reflection decisions, planner rules, validator checks.
`test_pipeline.py`	43	The full graph: agent order and data flow, persistence, solution protection, idempotency, failure handling.
`test_policy.py`	30	DSDA score, levels, ladders, timing and the documented worked example.
`test_reflection_and_planner.py`	27	Reflection questions and replies, their effects, and planner sequencing and agreement.
`test_hint_quality.py`	23	Specific, non-generic hint wording and validator behaviour.
`test_content.py`	13	Curriculum integrity: lessons, techniques and test coverage.

User instructions in brief. A learner chooses a topic or the recommended next step, reads the lesson if needed, writes code in the workspace, uses Run to try visible examples, and Submit to be graded. The tutor responds after a submission or when the learner presses "Ask for a hint"; "I need more help" asks for the next step, and selecting lines lets the learner ask about them. A teacher creates a class, shares its join code, and follows learners on the class dashboard.

Additional screens. Figures A.1 and A.2 show a topic lesson and the practice library.

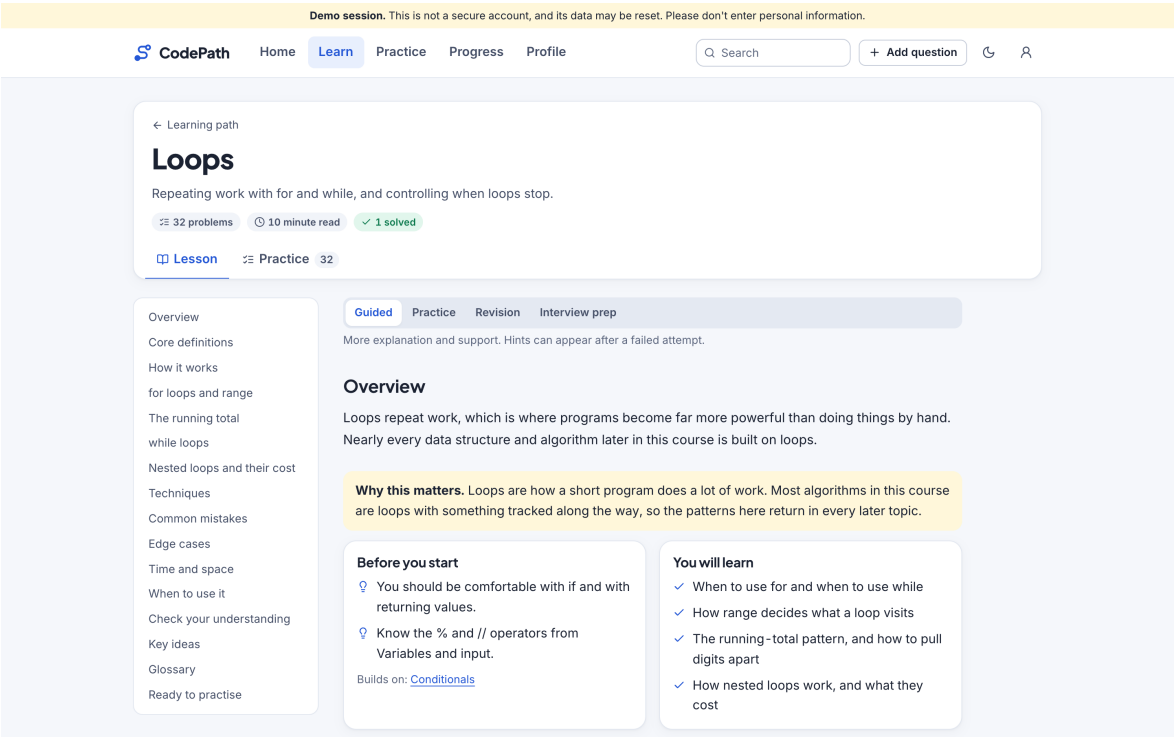


Figure A.1: A topic lesson, opened from the problem's "Read the lesson" link

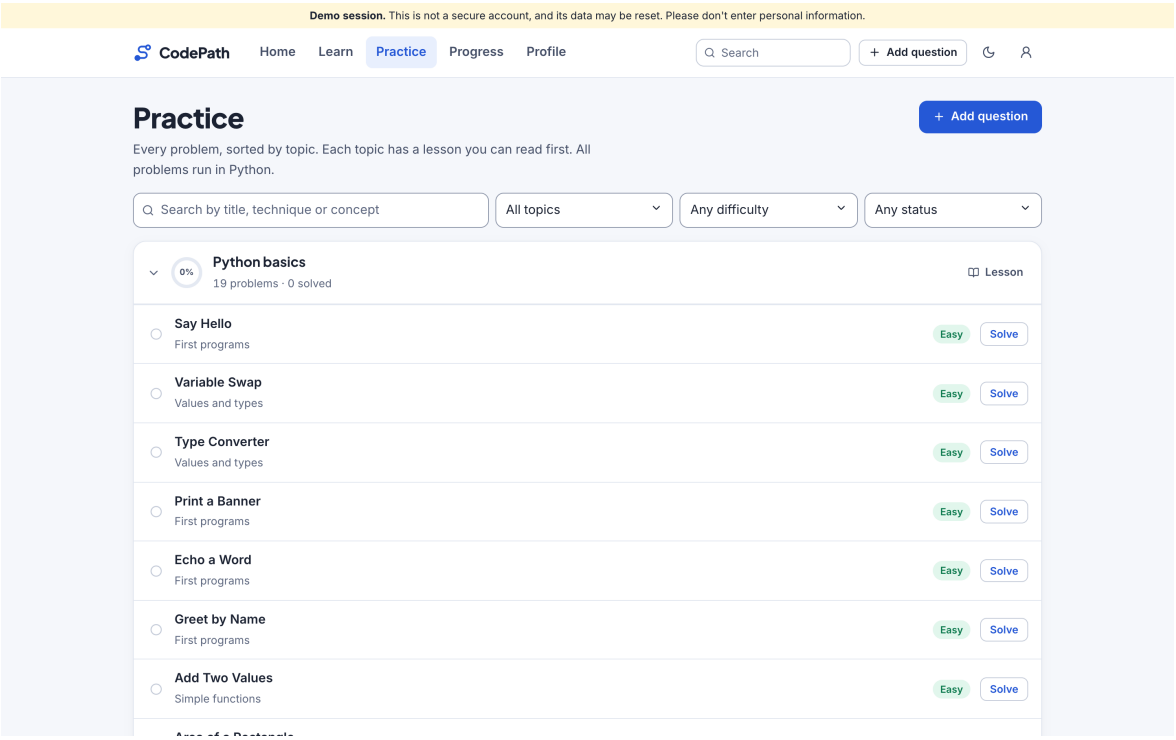


Figure A.2: The practice library, grouping problems by topic in learning order